МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ, НГУ)

Факультет Естественных Наук

Кафедра Информационная биология

Направление подготовки 06.04.01 Биология

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА МАГИСТРА

Карманова Сергея

Тема работы:

ПРЕДСКАЗАНИЕ EC-НОМЕРА ФЕРМЕНТА НА ОСНОВЕ МЕТОДОВ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА С УЧЕТОМ ОПЕРОННОЙ СТРУКТУРЫ ГЕНОМОВ ПРОКАРИОТ

«К защите допущен»	**Научный руководитель**
Заведующий кафедрой,	К.т.н., доцент
Д.б.н., профессор, академик РАН	Н. с., ИЦиГ СО РАН
Н.А. Колчанов	П.С.Деменков
_____	_____
«____» мая 2026 г	«____» мая 2026 г

Новосибирск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ....4
1 ОБЗОР ЛИТЕРАТУРЫ....6
1.1 Проблема функциональной аннотации белков....6
1.2 Оперонная организация генов прокариот....8
1.2.1 Определение и общая архитектура оперона....8
1.2.2 Функциональная кластеризация и координированная регуляция....9
1.2.3 Классические примеры оперонной организации....10
1.2.4 Особенности оперонной организации и применение в биоинформатике....11
1.3 Метаболические пути и их роль в клеточных процессах....11
1.4 Классификация ферментов....14
1.5 Методы предсказания функций белков....16
1.5.2 Методы машинного обучения....17
1.5.3 Модели глубокого обучения для предсказания функций белков....19
1.5.4 Модель ProteInfer....20
1.5.5 Модель CLEAN....21
2 ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ....23
2.1 Предобработка данных Operon DataBase....23
2.2 Выбор референсных сборок и извлечение идентификаторов белков....24
2.3 Разбор GFF-файлов и построение таблицы соответствия....25
2.4 Загрузка и фильтрация данных UniProt....26
2.5 Итоговое объединение и характеристики объединённого набора....27
2.6 Подготовка оценочного набора....29
2.7 Обогащение метаболическими путями KEGG....29
2.8 Стратификация оценочного набора по уровню гомологии....30
2.9 Анализ степени пересечения метаболических путей KEGG в оперонах....30
2.10 Оперонный фильтр....32
2.11 Метрики оценки качества предсказаний....33
2.12 Выбор оптимального порога уверенности....34
3 ОБСУЖДЕНИЕ РЕЗУЛЬТАТОВ....36
3.1 Характеристики оценочного набора и функциональная однородность оперонов 36
3.2 Базовая оценка моделей и выбор оптимального порога достоверности....38
3.3 Эффективность оперонного фильтра....41

3.4 Независимость уверенности модели и оперонного контекста 45
ВЫВОДЫ 47
СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 48
СПИСОК ЦИТИРОВАННОЙ ЛИТЕРАТУРЫ 49
ПРИЛОЖЕНИЕ 54

ВВЕДЕНИЕ

Современные технологии секвенирования нового поколения (NGS) позволили исследователям получить доступ к огромному объему генетической информации, включая геномы и транскриптомы различных организмов. Однако, с увеличением количества доступных белковых последовательностей, особенно из немодельных организмов, возникает проблема аннотации этих последовательностей. Стандартные биоинформатические методы, основанные на сходстве, могут оставлять до 50% предполагаемых последовательностей, кодирующих белок, неаннотированными, что затрудняет понимание их функций [1].

В данной работе рассматривается прогнозирование класса ферментов на основе анализа данных о опероннах и последовательностях белков с применением различных методов глубокого обучения. Исследование проводится с целью улучшения точности аннотации белков и предсказания их функций. Для этого используются различные базы данных, включая Operon DataBase (ODB), Kyoto Encyclopedia of Genes and Genomes (KEGG) [3].

Цель работы – разработать и обосновать метод повышения точности предсказания полных четырёхуровневых EC-номеров прокариотических ферментов на основе верификации предсказаний моделей глубокого обучения с использованием оперонного функционального контекста с оценкой вклада данного подхода в условиях различной степени гомологии к обучающим данным:

1. Сформировать репрезентативный набор данных, объединяющий информацию об аминокислотных последовательностях ферментов, их аннотациях EC-номерами и метаболическими путями KEGG, а также информацию о расположении генов, кодирующих эти ферменты, в оперонной структуре генома.
2. Количественно оценить степень пересечения метаболических путей KEGG у генов одного оперона для обоснования применимости оперонного контекста в качестве критерия верификации предсказаний.
3. Провести сравнительную оценку существующих моделей глубокого обучения для предсказания EC-номеров на подготовленном наборе данных и объективно определить оптимальные пороги параметров моделей
4. Разработать метод фильтрации результатов предсказания EC-номеров ферментов по аминокислотным последовательностям с учётом оперонной структуры генома и оценить его эффективность по стратам гомологии и уровням EC-иерархии

5. Оценить вклад оперонного контекста в повышение качества предсказаний модели глубокого обучения по стратам гомологии

1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Проблема функциональной аннотации белков

Развитие технологий секвенирования нового поколения (NGS) в последние два десятилетия привело к многократному снижению стоимости и росту точности определения нуклеотидных последовательностей геномов и транскриптомов различных организмов. Благодаря этим технологиям общедоступные биологические базы данных ежегодно пополняются сотнями миллионов новых последовательностей, значительную часть которых составляют последовательности прокариотических белков из так называемых немодельных организмов – видов, не являющихся традиционными объектами экспериментальных исследований. Лавинообразный рост объёма первичных данных создаёт принципиальный дисбаланс: объём генерируемых последовательностей растёт значительно быстрее, чем возможности их функциональной характеризации экспериментальными методами.

Количественный масштаб задачи иллюстрирует динамика роста объёма базы данных UniProt Knowledgebase (UniProtKB). По состоянию на 2025 год она содержит около 246 млн последовательностей белков [4], при этом лишь примерно 0,2 % из них (около 575 тыс. записей UniProtKB/Swiss-Prot) прошли ручную экспертную проверку, а оставшаяся подавляющая часть аннотирована автоматическими методами в составе UniProtKB/TrEMBL [4]. Среди рецензированных белков порядка 45 % описывают ферменты [5], что подчёркивает центральную роль каталитически активных белков в задаче функциональной аннотации и, в частности, обоснованность фокусировки на предсказании EC-номеров. Динамика накопления последовательностей в UniProtKB за период с 2012 по 2022 год представлена на рисунке 1.

Ключевой задачей при обработке этих данных является функциональная аннотация – присвоение каждому белку указания на его биологическую функцию, в частности, классификационного номера фермента для каталитически активных белков. По существующим оценкам до 50 % предсказанных белок-кодирующих последовательностей в новых геномах остаются функционально неаннотированными, а значительная часть биохимически важных белков известных семейств ещё не охарактеризована экспериментально [1]. Неполнота функциональных аннотаций существенно ограничивает возможности биологической интерпретации геномных данных и препятствует пониманию метаболических и регуляторных сетей в исследуемых организмах.

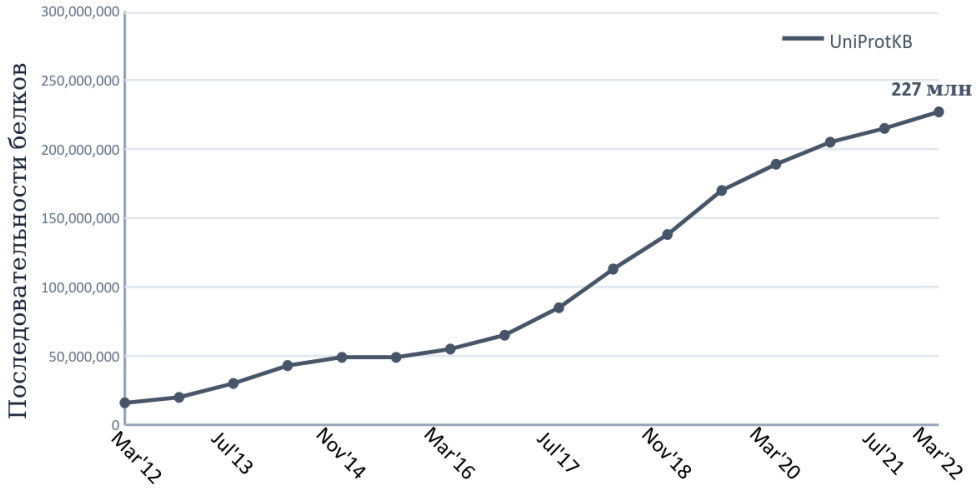


Рисунок 1 – рост числа последовательностей белков в базе данных UniProtKB с марта 2012 по март 2022 года [3].

Стандартным подходом к автоматической функциональной аннотации белков является поиск по сходству последовательностей: белку присваивается функция наиболее близкого по аминокислотной последовательности гомологичного белка, для которого эта функция была ранее установлена экспериментально или предсказана. Основным инструментом этого подхода является алгоритм Basic Local Alignment Search Tool (BLAST) [6], в течение многих лет остающийся главным средством гомологического поиска в геномике и протеомике. Несмотря на свою высокую эффективность для белков с близкими гомологами в референсных базах данных, подход на основе гомологии имеет ряд принципиальных ограничений.

Первое ограничение связано с явлением функционального дрейфа – постепенного изменения биологической функции в ходе эволюции белков при сохранении высокого сходства их аминокислотных последовательностей. Белок, унаследовавший 60–70 % идентичности от общего предка, может приобрести изменённую субстратную специфичность, сменить кофактор или перейти к выполнению иной каталитической реакции, оставаясь при этом «похожим» на исходный белок с точки зрения алгоритмов выравнивания. В результате перенос функциональной аннотации по гомологии может приводить к систематическим ошибкам, которые затем распространяются при последующих циклах аннотации новых геномов на основе уже накопленных данных; количественный анализ публичных баз данных показал, что доля таких ошибок в архивных базах NR, TrEMBL и KEGG накапливалась в период с 1993 по 2005 год [1].

В исследовании Schnoes et al. на выборке из 37 экспериментально охарактеризованных семейств ферментов из шести суперсемейств было показано, что

средний уровень ошибочной аннотации в архивных базах GenBank NR, UniProtKB/TrEMBL и KEGG составляет от 5 до 63 % в зависимости от семейства, а для 10 семейств из 37 уровень ошибок превышает 80 % [1]. Типичным видом ошибки является так называемая переспецификация – присвоение белку конкретной каталитической функции при отсутствии достаточных оснований, тогда как аннотация только на уровне общего класса или суперсемейства была бы более корректной. Второе принципиальное ограничение метода заключается в том, что при низкой гомологии (менее 30–40 % идентичности) алгоритмы выравнивания теряют чувствительность и специфичность, что делает их неэффективными для аннотации белков из эволюционно удалённых или плохо изученных таксономических групп. Третье ограничение носит системный характер: методы на основе гомологии принципиально не способны предсказать функцию для белков, не имеющих охарактеризованных аналогов в обучающих базах данных.

Совокупность перечисленных ограничений обуславливает интерес к альтернативным методам функциональной аннотации, основанным на машинном обучении и, в частности, на глубоких нейронных сетях. Крупномасштабное сравнение 54 вычислительных методов предсказания функций белков в рамках проекта Critical Assessment of Function Annotation (CAFA) показало, что наилучшие из ранних методов машинного обучения существенно превосходят по точности традиционные подходы на основе BLAST, что стимулировало активное развитие нейросетевых архитектур в этой области [7]. Вместе с тем и методы глубокого обучения не свободны от ошибок: при анализе низкогомологичных белков их предсказания могут быть уверенными, но некорректными. Это мотивирует поиск дополнительных источников информации, не сводимых к аминокислотной последовательности, таких как геномный контекст, эволюционная консервативность, структурные особенности и межбелковые взаимодействия. Одним из наиболее перспективных источников такой информации для прокариотических белков является опероннная организация генов, рассматриваемая в последующих разделах настоящего обзора.

1.2 Опероннная организация генов прокариот

1.2.1 Определение и общая архитектура оперона

Опероны прокариот представляют собой кластеры генов, транскрибируемых совместно в виде единой полицистронной мРНК. Такая организация принципиально отличает бактерии и археи от эукариот, у которых каждый ген обычно имеет собственный

промотор. В состав оперона входят несколько обязательных цис-регуляторных элементов: промотор, оператор и терминатор [8].

Промотор представляет собой участок связывания РНК-полимеразы и σ-фактора; сила промотора определяет базовый уровень транскрипции. Оператор – это короткая последовательность ДНК, с которой связывается регуляторный белок (репрессор или активатор); перекрывание оператора с промотором или сайтом инициации транскрипции блокирует синтез мРНК. Терминатор – сигнал окончания транскрипции (ρ-зависимый или ρ-независимый, формирующий шпильку).

Мутации в промоторе (например, в lacP) снижают сродство к РНК-полимеразе, делая оперон конститутивно слабым; мутации в операторе (lacO) нарушают связывание репрессора, вызывая постоянную экспрессию даже в отсутствие индуктора.

1.2.2 Функциональная кластеризация и координированная регуляция

Проведение пространственного анализа расположения генов в геноме позволило выявить корреляции, предполагающие наличие функциональных связей между кодируемыми белками. Белки, принимающие участие в аналогичных метаболических процессах, зачастую находятся в непосредственной близости друг от друга в составе одного оперона, что свидетельствует о возможной общей функциональной активности или участии в последовательных биохимических превращениях [9]. Более того, компактная упаковка генов в опероны облегчает горизонтальный перенос (например, кластеры генов устойчивости к антибиотикам часто организованы как единые опероны на плазмидах). Эволюционно такая структура снижает риск разделения функционально связанных локусов при рекомбинации.

Оперонная регуляция обеспечивает эффективное использование ресурсов клетки путём координированного синтеза белков в ответ на изменения внешней среды. Вместо того чтобы запускать транскрипцию десяти генов по отдельности, клетка синтезирует одну полицистронную мРНК, что экономит АТФ на инициацию и позволяет рибосомам одновременно транслировать несколько белков. При этом возможны полярные эффекты: нонсенс-мутация в раннем гене оперона вызывает преждевременную терминацию трансляции и, из-за отсутствия защиты мРНК от рибонуклеаз, деградацию всей последующей части транскрипта.

1.2.3 Классические примеры оперонной организации

Примерами хорошо изученных оперонов являются лактозный оперон (lac, контролирующий метаболизм лактозы) и триптофановый оперон (trp, ответственный за биосинтез триптофана) [10].

Лактозный оперон, представленный у *E. coli* работает следующим образом. В отсутствие лактозы репрессор LacI (тетрамер) связывается с оператором и изгибает ДНК, блокируя РНК-полимеразу. При наличии лактозы клетка превращает её в аллолактозу – индуктор, который связывается с LacI, меняет конформацию репрессора, снижая его сродство к оператору. Дополнительно механизм катаболитной репрессии через CAP-цАМФ-комплекс обеспечивает приоритетное использование глюкозы.

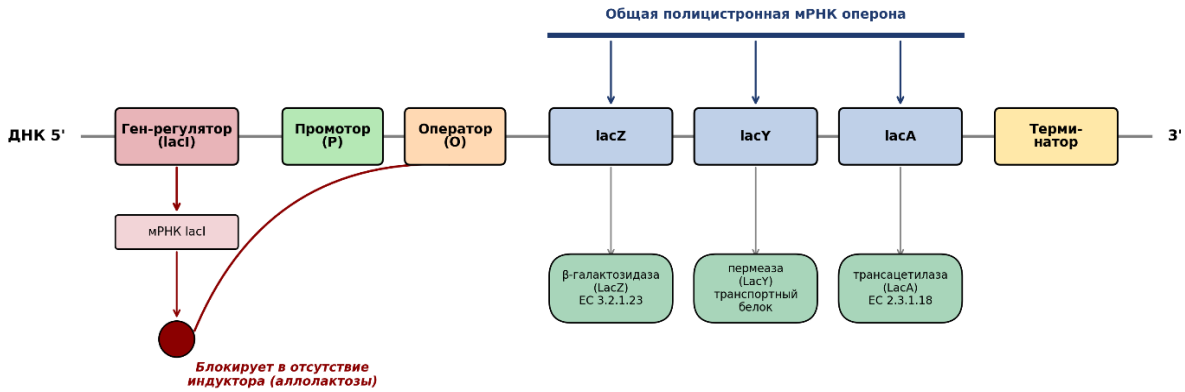


Рисунок 2 – схема оперонной структуры генома прокариот на примере lac-оперона *E. coli.*

Принцип действия триптофанового оперона следующий. При избытке триптофана апорепрессор TrpR связывает триптофан (корепрессор), активируется и присоединяется к оператору, подавляя транскрипцию. Основная тонкая настройка происходит через механизм аттенюации. В лидерном пептиде (последовательность из 14 аминокислот с двумя соседними триптофанами) рибосома, транслирующая этот короткий пептид, при высоком уровне триптофанил-тРНК быстро доходит до стоп-кодона, что позволяет сформироваться терминаторной шпильке (аттенюатору) на уровне РНК. При дефиците триптофана рибосома задерживается, формируется антитерминаторная структура, и РНК-полимераза продолжает синтез полного оперона. Это пример регуляции без участия репрессора, напрямую зависящий от загрузки тРНК.

1.2.4 Особенности оперонной организации и применение в биоинформатике

Не все соседние гены образуют единый оперон. Внутри некоторых оперонов обнаружены внутренние промоторы (например, в gal-опероне *E. coli*) [10], позволяющие транскрибировать часть генов независимо. Также существуют «частичные опероны», где гены разбиты на две транскрипционные единицы, но функционально связаны. При аннотации геномов важно отличать истинные опероны от случайных сцеплений генов.

Понимание принципов организации оперонов способствует более точной аннотации генов и предсказанию их функций. Если ген с неизвестной функцией находится в составе оперона, содержащего гены с известными функциями, можно предположить, что этот ген также участвует в данном или связанном метаболическом пути. Этот принцип применяется в современных методах биоинформатики для аннотирования генов и предсказания их функций [11].

Современные базы данных и веб-ресурсы, например, Operon DataBase (ODB), Database of prOkaryotic OpeRons (DOOR), Prediction of Prokaryotic Promoter Elements and Regulons (PePPER) используют комплекс признаков: межгенные расстояния (у оперонов они обычно составляют менее 50–60 п.н., тогда как у независимых генов существенно больше), наличие терминаторов и промоторов, данные сравнительной геномики (консервативность соседства генов в нескольких видах), а также обучаемые модели (скрытые марковские модели, случайные леса), объединяющие длины генов, сходство GC-состава и данные ко-экспрессии по RNA-seq [12,13].

На практике для неизвестного гена, расположенного в предполагаемом опероне с известными ферментами пути метаболизма аминокислот, можно с высокой уверенностью отнести его к тому же пути ещё до экспериментальной проверки [14]. Это стало рутинной процедурой при автоматической аннотации бактериальных геномов.

1.3 Метаболические пути и их роль в клеточных процессах

Метаболические пути (МП) представляют собой упорядоченные каскады биохимических реакций, протекающих внутри клеток и катализируемых ферментами. Они играют центральную роль в поддержании гомеостаза и жизнеспособности клеток, обеспечивая трансформацию молекул, синтез функциональных макромолекул, производство энергии и элиминацию продуктов метаболизма [15]. Каждая реакция в метаболическом пути представляет собой звено в цепи последовательных превращений, где продукт одной ферментативной реакции служит субстратом для следующей. Совокупность

таких звеньев формирует целостный маршрут, ведущий от исходного субстрата к конечному продукту, необходимому клетке для выполнения конкретной биологической функции.

Метаболические пути делятся на три основных типа: катаболические, анаболические и амфиболические [16]. Катаболические пути направлены на расщепление сложных органических молекул до более простых соединений с высвобождением энергии, запасаемой преимущественно в форме АТФ и восстановленных коферментов (НАДН, $ФАДН_2$). Классическими примерами катаболизма являются гликолиз, β-окисление жирных кислот и цикл трикарбоновых кислот (цикл Кребса). Анаболические пути, напротив, обеспечивают синтез сложных макромолекул из более простых предшественников с затратой энергии, к ним относятся глюконеогенез, биосинтез аминокислот, нуклеотидов и липидов. Амфиболические пути занимают промежуточное положение и могут функционировать как в катаболическом, так и в анаболическом направлении в зависимости от метаболических потребностей клетки; наиболее известным примером является цикл трикарбоновых кислот, промежуточные соединения которого одновременно окисляются для получения энергии и служат предшественниками для биосинтетических реакций.

Метаболические пути обладают рядом характерных свойств, обеспечивающих их эффективное функционирование в клетке.

Необратимость ключевых реакций означает, что отдельные этапы пути являются термодинамически направленными в одну сторону и выступают в качестве регуляторных точек. Такие реакции, катализируемые регуляторными ферментами (например, фосфофруктокиназой в гликолизе или изоцитратдегидрогеназой в цикле Кребса), определяют общий поток метаболитов через путь и не могут быть обращены вспять без участия альтернативных ферментов [17].

Стадийность метаболических путей проявляется в том, что химическое превращение исходного субстрата в конечный продукт происходит не в одну реакцию, а поэтапно, через серию промежуточных соединений. Каждый этап катализируется специфическим ферментом, что позволяет клетке осуществлять сложные химические превращения при мягких условиях (физиологические температура и pH) и с высокой селективностью. Стадийность также даёт возможность тонко регулировать скорость пути на каждом из этапов, а интермедиаты могут служить точками ветвления, направляя метаболиты в альтернативные пути в зависимости от текущих потребностей клетки.

Регулируемость метаболизма обеспечивается через множество механизмов контроля активности ферментов. К основным способам регуляции относятся аллостерическая

модуляция (связывание эффектора с некаталитическим сайтом фермента, вызывающее конформационные изменения и изменение активности), ковалентная модификация (например, фосфорилирование и дефосфорилирование ферментов протеинкиназами и фосфатазами), регуляция на уровне экспрессии генов (изменение количества фермента в клетке), а также контроль через энергетическое состояние клетки – соотношение АТФ/АДФ и $НАД^+$/НАДН [14].

Взаимосвязь метаболических путей выражается в том, что они функционируют не изолированно, а как интегрированные сети, образующие единую метаболическую карту клетки. Продукты одного пути регулярно выступают субстратами или регуляторами другого, формируя многоуровневую систему перекрёстной регуляции. Например, ацетил-КоА, образующийся в результате β-окисления жирных кислот и окислительного декарбоксилирования пирувата, поступает в цикл Кребса, а интермедиаты этого цикла используются для синтеза аминокислот, порфиринов и глюкозы [16]. Такая архитектура позволяет клетке гибко перераспределять ресурсы и адаптироваться к изменяющимся условиям среды.

Понимание организации метаболических путей имеет непосредственное значение для биоинформатики и компьютерного предсказания функций белков. Современные подходы к функциональной аннотации используют различные источники информации помимо аминокислотной последовательности. Модели глубокого обучения, объединяющие анализ последовательности с данными о сетях белок-белковых взаимодействий, способны выявлять функциональные связи, недоступные при анализе одной лишь последовательности: в частности, модель DeepGO продемонстрировала, что интеграция последовательностных признаков с топологией сетей белок-белковых взаимодействий повышает точность предсказания терминов [18]. Другой подход основан на использовании пространственной структуры белка: графовые свёрточные сети, обученные на контактных картах белков, позволяют предсказывать функцию на основе трёхмерной организации аминокислотных остатков, что особенно ценно для белков с низкой гомологией к охарактеризованным ферментам [19].

Дополнительным источником функциональной информации для прокариотических белков является геномный контекст. Ещё в конце 1990-х годов было показано, что консервативное соседство генов в геномах различных организмов свидетельствует об их функциональной связи и может использоваться для предсказания функций неохарактеризованных генов [20]. Развитием этой идеи стали методы, использующие опероннную организацию генов у прокариот, взаимное расположение генов на хромосоме и

паттерны ко-экспрессии для переноса функциональных аннотаций [21]. Если ген неизвестной функции расположен в составе оперона, содержащего гены с установленной принадлежностью к определённому метаболическому пути, это позволяет с высокой вероятностью отнести кодируемый им белок к тому же или функционально связанному пути. Подобный подход, сочетающий анализ последовательности, структуры и геномного контекста, лежит в основе современных методов функциональной аннотации геномов и активно применяется в крупномасштабных проектах по изучению метагеномов, где значительная доля обнаруженных генов не имеет экспериментально подтверждённой функции.

1.4 Классификация ферментов

Каждому ферменту в современной биохимии присваивается уникальный классификационный номер в соответствии с номенклатурой Enzyme Comission (EC) разработанной International Union of Biochemistry and Molecular Biology (IUBMB) [22]. Данная система представляет собой цифровую иерархию, в которой каждый фермент получает четырёхуровневый код, состоящий из цифр, разделённых точками (например, 1.1.1.1). Эти цифры не являются произвольными – они последовательно отражают специфику катализируемой реакции, тип химического превращения, участвующие субстраты и кофакторы, а также уникальное положение фермента в общей классификации [23]. Структура EC-номера включает четыре уровня детализации, каждый из которых уточняет предыдущий.

Первая цифра (класс) определяет принадлежность фермента к одному из семи основных классов в зависимости от общего типа катализируемой химической реакции. EC 1 – оксидоредуктазы, катализирующие окислительно-восстановительные реакции и перенос электронов (дегидрогеназы, оксидазы, пероксидазы). EC 2 – трансферазы, переносящие химические группы (метильные, фосфатные или аминогруппы) с одного субстрата на другой. EC 3 – гидролазы, катализирующие гидролитическое расщепление связей с участием воды (протеазы, эстеразы, нуклеазы). EC 4 – лиазы, разрывающие связи негидролитическим путём, часто с образованием двойной связи или присоединением группы к двойной связи. EC 5 – изомеразы, катализирующие внутримолекулярные перестройки, то есть превращение одного изомера в другой. EC 6 – лигазы, обеспечивающие соединение двух молекул за счёт гидролиза АТФ или другого

нуклеозидтрифосфата. EC 7 – транслоказы, переносящие ионы или молекулы через мембраны, сопрягая этот процесс с гидролизом АТФ.

Вторая цифра (подкласс) уточняет характер реакции внутри основного класса, часто указывая на тип донорной или акцепторной группы, вовлечённой в процесс. Например, для оксидоредуктаз (EC 1) вторая цифра обозначает донор электронов: EC 1.1 – спиртовые группы, EC 1.2 – альдегидные или кетогруппы, EC 1.3 – CH–CH-группы. Для трансфераз (EC 2) вторая цифра указывает на тип переносимой группы: EC 2.1 – одноуглеродные группы, EC 2.3 – ацильные группы, EC 2.7 – фосфатные группы (киназы).

Третья цифра (подподкласс) ещё точнее характеризует реакцию, конкретизируя акцептор, участвующую связь, специфический реагент или конечный продукт. Например, для алкогольдегидрогеназы (EC 1.1.1.1): EC 1 (класс: оксидоредуктазы) → EC 1.1 (подкласс: действие на донорную группу CH–OH) → EC 1.1.1 (подподкласс: акцептор – $НАД^+$ или $НАДФ^+$).

Четвёртая цифра (уникальный идентификатор) служит для окончательной идентификации конкретного фермента внутри подподкласса. Эта цифра отражает тонкие каталитические особенности: специфичность к определённому субстрату, тип кофактора (например, металл, $НАД^+$, ФАД, пиридоксальфосфат), а также участие конкретных метаболических интермедиатов. Например, в подподклассе 1.1.1 (оксидоредуктазы, действующие на CH–OH-группу донора с $НАД^+$/$НАДФ^+$ в качестве акцептора) имеются: EC 1.1.1.1 – алкогольдегидрогеназа, окисляющая первичные и вторичные спирты; EC 1.1.1.27 – лактатдегидрогеназа, окисляющая L-лактат до пирувата; EC 1.1.1.37 – малатдегидрогеназа, окисляющая L-малат до оксалоацетата.

Таким образом, каждая последующая цифра EC-номера сужает предыдущий уровень, обеспечивая однозначную идентификацию фермента среди тысяч известных. Благодаря этой системе достигается унификация биохимических данных: EC-номер выступает как международный стандарт для поиска информации в базах данных BRaunschweig ENzyme DAtabase (BRENDA), KEGG, UniProt, MetaCyc [24], сравнения ферментов разных организмов, а также для предсказания функции неизученных ферментов на основе гомологии [25]. В научных публикациях указание EC-номера является обязательным при первом упоминании фермента, что исключает неоднозначность, вызванную тривиальными или синонимичными названиями.

1.5 Методы предсказания функций белков

1.5.1 Подходы на основе гомологии и ортологии

Исторически доминирующим подходом к автоматической функциональной аннотации белков является перенос аннотации по гомологии: для неохарактеризованного белка находится наиболее сходный по аминокислотной последовательности белок с известной функцией, после чего его аннотация переносится на запрос. Основным инструментом такого поиска является алгоритм BLAST [6]. BLAST выполняет эвристическое локальное выравнивание запроса против базы данных последовательностей и оценивает статистическую значимость найденных совпадений на основе модели максимальных сегментных пар. Благодаря высокой скорости и простоте интерпретации результатов BLAST остаётся одним из наиболее цитируемых биоинформатических инструментов.

Развитием подходов на основе сходства последовательностей являются методы анализа ортологии, в которых для предсказания функции используется не одиночное попарное сходство, а принадлежность белка к группе ортологов – белков различных организмов, происходящих от общего предкового гена и сохраняющих, как правило, функциональную идентичность. К наиболее известным инструментам данной категории относятся Ortholog Groups of Protein Sequences (OrthoMCL) [26] и Orthologous Matrix (OMA) [27]. OrthoMCL основан на алгоритме Markov Clustering, который группирует белки в ортологические группы на основе матрицы взаимных лучших совпадений, полученных попарным сравнением последовательностей с помощью BLAST. OMA реализует более строгий подход к определению ортологии, основанный на поиске взаимных лучших совпадений между парами геномов и их последующем объединении в иерархические ортологические группы; в обновлении 2024 года база OMA была существенно расширена в сторону прокариотических геномов. При работе с неохарактеризованным белком его относят к одной из заранее построенных ортологических групп и переносят на него аннотацию, разделяемую большинством членов группы.

Подходы на основе гомологии и ортологии имеют ряд принципиальных ограничений. Первое связано с явлением функционального дрейфа – постепенного изменения биологической функции в ходе эволюции белков при сохранении высокого сходства их аминокислотных последовательностей. Белок, унаследовавший 60–70 % идентичности от общего предка, может приобрести изменённую субстратную специфичность, сменить

кофактор или перейти к выполнению иной каталитической реакции, оставаясь при этом «похожим» на исходный белок с точки зрения алгоритмов выравнивания. В результате перенос функциональной аннотации может приводить к систематическим ошибкам; количественный анализ публичных баз данных показал, что доля таких ошибок в архивных базах GenBank NR, TrEMBL и KEGG достигает 5–63 %, а для отдельных семейств ферментов превышает 80 % [1]. Второе ограничение заключается в том, что при низкой гомологии (менее 30–40 % идентичности) алгоритмы выравнивания теряют чувствительность и специфичность, что делает их неэффективными для аннотации белков из эволюционно удалённых или плохо изученных таксономических групп. Третье ограничение носит системный характер: методы на основе гомологии и ортологии принципиально не способны предсказать функцию для белков, не имеющих охарактеризованных аналогов в обучающих базах данных.

Совокупность перечисленных ограничений обуславливает интерес к альтернативным подходам, не зависящим напрямую от наличия охарактеризованного гомолога – методам на основе классического машинного обучения и глубоких нейронных сетей, рассмотренным в последующих разделах.

1.5.2 Методы машинного обучения

Принципиальным шагом в развитии методов функциональной аннотации стал переход от подходов на основе гомологии к задаче классификации белков средствами машинного обучения. В рамках этого подхода аминокислотная последовательность преобразуется в числовое представление – вектор физико-химических, статистических или композиционных дескрипторов, на котором затем обучается алгоритм классификации, отображающий вектор признаков в один или несколько функциональных классов. В отличие от методов на основе гомологии, такие алгоритмы не требуют наличия близкого охарактеризованного гомолога в обучающей выборке и в принципе способны выявлять закономерности, не сводимые к попарному сходству последовательностей.

Одним из ранних представителей данного направления является SVM-Prot [28] – веб-сервис, классифицирующий белки на функциональные семейства с помощью метода опорных векторов (SVM) на основе физико-химических дескрипторов всей последовательности, включая аминокислотный состав, гидрофобность, поляризуемость, заряд и предсказанную вторичную структуру. SVM-Prot реализует независимый бинарный SVM-классификатор для каждого функционального семейства, что позволяет относить

запросный белок к произвольному числу семейств одновременно. Изначально метод охватывал 54 функциональных семейства; в последующих обновлениях это число было расширено до 192. SVM-Prot не предсказывает EC-номера напрямую, однако концептуально близкие подходы были позднее адаптированы непосредственно к задаче классификации ферментов.

Специализированным инструментом машинного обучения для предсказания EC-номеров является ECPred [29], построенный как ансамбль из трёх независимых классификаторов: первый основан на гомологическом поиске BLAST, второй на сходстве субсегментов последовательности, третий на физико-химических дескрипторах пептида в духе SVM-Prot. Окончательное решение формируется агрегацией предсказаний отдельных классификаторов через взвешенное голосование. Для каждого EC-номера обучается независимая модель с собственным порогом отсечения, что позволяет проводить оптимизацию параметров классификации индивидуально для каждой ферментативной функции. Обучение ECPred выполнялось на курируемой части базы UniProtKB/Swiss-Prot, и инструмент предоставляет иерархические предсказания на всех четырёх уровнях EC-номенклатуры.

Несмотря на принципиальное преимущество перед чисто гомологическими методами в способности обобщать на белки без близких аналогов в обучающей выборке, классические подходы на основе машинного обучения демонстрируют умеренную точность предсказания полных четырёхуровневых EC-номеров. В работе [30] на двух независимых тестовых наборах: наборе New-392, включающем 392 фермента 177 различных EC-номеров из SwissProt, и наборе Price-149, состоящем из 149 экспериментально валидированных бактериальных белков, точность предсказания EC-номеров методом ECPred составила 14 % на всех организмах и 4 % на прокариотах. Существенное падение точности на прокариотическом бенчмарке Price-149 отражает фундаментальное ограничение методов, основанных на интегральных дескрипторах всей последовательности: расположение остатков активного центра, во многом определяющее каталитическую специфичность фермента, в признаки такого рода практически не попадает. Это обстоятельство стало одной из основных мотиваций для перехода к моделям глубокого обучения, способным самостоятельно выявлять локальные мотивы и комбинации остатков, значимые для функции.

1.5.3 Модели глубокого обучения для предсказания функций белков

Применение методов машинного обучения к задаче предсказания функций белков началось с классических подходов (случайные леса, метод опорных векторов), но существенный прорыв произошёл с появлением глубоких нейросетевых архитектур. Крупномасштабное сравнение 54 вычислительных методов предсказания функций белков, проведённое в рамках международного проекта Critical Assessment of Function Annotation (CAFA), показало, что лучшие из ранних методов машинного обучения превосходят традиционные подходы на основе BLAST по точности предсказания функциональных категорий [7]. Последующие конкурсы CAFA подтвердили эту тенденцию и стимулировали активное развитие нейросетевых архитектур.

Современные нейросетевые модели предсказания функций белков можно условно разделить на три класса по типу используемых архитектур. Свёрточные нейронные сети (CNN) обрабатывают аминокислотную последовательность как одномерный сигнал, выявляя локальные паттерны (мотивы), значимые для определения функции [31]. Рекуррентные и трансформерные модели анализируют последовательность как упорядоченный текст, улавливая дальнодействующие зависимости между остатками. Графовые нейронные сети (GNN) работают с пространственной структурой белка или с сетями межбелковых взаимодействий. Все эти подходы принимают на вход аминокислотную последовательность (или её производные – векторные представления), а на выход предоставляют вероятности принадлежности белка к функциональным категориям, таким как EC-номера или термины Gene Ontology (GO). [32].

Важным направлением является использование предобученных белковых языковых моделей, таких как ESM-1b, которые обучаются на сотнях миллионов белковых последовательностей по принципу маскированного моделирования языка и формируют контекстно-зависимые векторные представления каждого белка [33]. Эти представления затем используются в качестве входных данных для специализированных моделей классификации, что позволяет переносить знания, извлечённые из эволюционного разнообразия белков, на конкретные задачи предсказания функций.

В настоящей работе используются две нейросетевые модели предсказания EC-номеров, представляющие два принципиально различных подхода: ProteInfer, основанная на свёрточных нейронных сетях, и Contrastive Learning-Enabled Enzyme Annotation (CLEAN), основанная на контрастивном обучении. Выбор этих моделей обусловлен тем, что они отражают два магистральных направления в данной области: прямую

классификацию по последовательности с помощьюмодели ProteInfer и метрическое обучение на векторных представлениях с помощью модели CLEAN. Подробное описание архитектуры и принципов работы каждой из моделей приведено ниже.

1.5.4 Модель ProteInfer

ProteInfer – нейросетевая модель предсказания функций белков, разработанная и реализованная в одноимённом программном пакете [34]. Модель принимает на вход необработанную аминокислотную последовательность произвольной длины и непосредственно предсказывает вероятности принадлежности белка к набору функциональных категорий, включая EC-номера ферментов и термины GO.

Архитектура ProteInfer представляет собой глубокую свёрточную нейронную сеть с расширенными свёртками [35] и остаточными связями [36]. Входная аминокислотная последовательность кодируется в виде one-hot матрицы и последовательно проходит через серию свёрточных слоёв. Каждый слой применяет набор фильтров, детектирующих паттерны признаков в последовательности. Использование остаточных связей облегчает обучение глубоких сетей, а расширенные свёртки позволяют каждому последующему слою охватывать всё более протяжённые участки входной последовательности без увеличения числа параметров. В результате верхние слои сети формируют представления, учитывающие контекст на масштабе сотен аминокислотных остатков [34].

После формирования позиционных векторных представлений модель выполняет их агрегацию в единый вектор фиксированной размерности с помощью операции усреднения. Такой подход позволяет обрабатывать последовательности произвольной длины без введения искусственных ограничений. Итоговый вектор подаётся на полносвязный слой с сигмоидальными активациями, выходы которого интерпретируются как независимые вероятности принадлежности белка к каждому из классов функциональных категорий. Обучение модели выполнялось на курируемой части базы данных UniProtKB/Swiss-Prot, что обеспечивает высокое качество обучающих аннотаций [34]. Ключевыми преимуществами ProteInfer являются вычислительная эффективность, отсутствие необходимости в предварительном выравнивании последовательностей и способность обеспечивать высокоточные предсказания, дополняющие результаты гомологического поиска. Вместе с тем, как и другие модели, обучающиеся непосредственно на аминокислотных последовательностях, ProteInfer подвержена снижению качества на белках с низкой гомологией к обучающей выборке.

На независимых тестовых наборах, использованных в работе [30] для сравнения существующих методов предсказания EC-номеров, точность предсказания полного четырёхуровневого EC-номера ProteInfer составила 31 % на наборе New-392 и 17 % на прокариотическом наборе Price-149 [30].

1.5.5 Модель CLEAN

CLEAN – модель предсказания EC-номеров ферментов, разработанная Yu et al. (2023) и опубликованная в журнале Science [30]. В отличие от ProteInfer, CLEAN использует парадигму метрического обучения: вместо прямой классификации белков по категориям модель обучается формировать векторные представления белков таким образом, чтобы ферменты с одинаковой функцией располагались близко в пространстве векторных представлений, а ферменты с различными функциями – далеко друг от друга.

На входе CLEAN использует векторные представления, полученные от предобученной белковой языковой модели ESM-1b, которая формирует контекстно-зависимые представления аминокислотных последовательностей на основе обучения на сотнях миллионов белков. Эти векторные представления подаются на нейронную сеть, обучаемую по принципу контрастивного обучения с учителем [37]: для каждого «якорного» белка формируются положительные примеры (белки с тем же EC-номером) и отрицательные примеры (белки с другими EC-номерами), после чего сеть минимизирует расстояния до положительных примеров и максимизирует расстояния до отрицательных [30].

Предсказание EC-номера для нового белка в CLEAN выполняется по принципу ближайшего центроида: для каждого EC-номера вычисляется центроид (среднее представление) всех белков обучающей выборки, аннотированных данным EC-номером; затем вычисляется евклидово расстояние от вектора запроса до каждого центроида, и белку присваивается EC-номер с наименьшим расстоянием. Предсказания оцениваются на основе разделения между расстоянием до ближайшего центроида и фоновым распределением расстояний [30].

Ключевым преимуществом CLEAN является способность эффективно аннотировать малоизученные ферменты, представленные немногочисленными примерами в обучающей выборке. Контрастивное обучение формирует устойчивые представления даже для редких EC-номеров, поскольку модель оптимизирует относительные расстояния, а не абсолютные вероятности принадлежности к классу. Авторы продемонстрировали, что CLEAN

превосходит BLAST по точности, надёжности и чувствительности, а также способен корректировать ошибочные аннотации и идентифицировать мультифункциональные ферменты с несколькими EC-номерами [30]. Вместе с тем, поскольку CLEAN стремится выдать предсказание для каждого белка на основе расстояния до ближайшего центроида, её предсказания при низкой confidence (уверенности) могут быть менее точными, чем у моделей с явным порогом отказа.

На тех же тестовых наборах, использованных для бенчмарка CLEAN с существующими методами, точность предсказания полного EC-номера составила 50 % на наборе New-392 и 49 % на прокариотическом наборе Price-149 [30]; в обоих случаях CLEAN существенно превзошёл альтернативные методы, в особенности на прокариотическом бенчмарке, где разрыв с ProteInfer достигал почти трёхкратной величины.

Таким образом, ProteInfer и CLEAN представляют два дополняющих друг друга подхода к предсказанию EC-номеров: ProteInfer обеспечивает высокую точность за счёт прямой свёрточной классификации последовательностей, тогда как CLEAN обеспечивает широкий охват и устойчивость к дисбалансу классов за счёт контрастивного метрического обучения. Различие архитектур и принципов обучения делает эти модели подходящими объектами для исследования того, насколько универсален эффект оперонного фильтра, предложенного в настоящей работе.

2 ЭКСПЕРИМЕНТАЛЬНАЯ ЧАСТЬ

2.1 Предобработка данных Operon DataBase

В качестве основного источника данных об оперонной организации генов была использована база данных ODB, которая аккумулирует сведения об известных и консервативных оперонах микроорганизмов. База данных объединяет как экспериментально подтверждённые опероны, описанные в научной литературе, так и предсказанные консервативные опероны, сформированные на основе ортологии: если гены известного оперона имеют ортологичные гены в другом геноме и эти ортологи последовательно расположены на одной и той же цепи ДНК, они рассматриваются как новый консервативный оперон. Текущая версия ODB содержит 6125841 запись об оперонах.

В работе использовалась версия ODB по состоянию на 25 ноября 2025 года (файл conserved_operon.download.txt). Обработка данных выполнялась средствами языка Python с применением библиотек pandas версии 1.3.5 и numpy версии 1.21.5.

Из исходного табулированного файла ODB (conserved_operon.download.txt) были извлечены три ключевых поля: идентификатор оперона (operon_id, соответствующий полю coid, формат CO000...), таксономический идентификатор организма (taxon_id, соответствующий идентификатору NCBI Taxonomy) и список генов, входящих в состав оперона. Каждая запись описывает конкретный оперон в рамках определённого организма; при этом один и тот же идентификатор оперона может встречаться в разных организмах, поэтому оперон однозначно определяется парой operon_id и taxon_id.

Предобработка данных выполнялась в три этапа. На первом этапе производилась нормализация структуры данных: списки генов каждого оперона развёртывались в отдельные записи (операция explode), так что каждая пара «оперон – ген» была представлена отдельной строкой. На втором этапе из набора удалялись записи с отсутствующими значениями в любом из трёх ключевых полей, а также дублирующиеся записи по комбинации всех трёх полей. На третьем этапе выполнялась фильтрация оперонов по размеру: записи, содержащие менее двух уникальных генов, исключались из набора данных, поскольку оперон по определению представляет собой группу из нескольких совместно транскрибируемых генов.

На заключительном этапе формировался список уникальных таксономических идентификаторов всех организмов, представленных в отфильтрованном наборе данных,

необходимый для последующего извлечения геномных аннотаций из базы данных NCBI RefSeq [38]. Итоговый набор данных был сохранён в формате Apache Parquet [39].

В результате предобработки исходный файл, содержавший 6125841 запись об оперонах, после развёртывания списков генов и удаления дубликатов был преобразован в нормализованный набор данных формата «один ген – одна строка», включавший 7963210 записей. После фильтрации оперонов с менее чем двумя генами итоговый набор содержал 3007902 записи, 1170533 уникальных оперона, 1174139 уникальных генов и 2711 уникальных таксономических идентификаторов организмов.

2.2 Выбор референсных сборок и извлечение идентификаторов белков

Данные ODB содержат идентификаторы генов в формате локус-тегов (locus tag), тогда как для последующего объединения с базой данных UniProt, содержащей аминокислотные последовательности и аннотации EC-номеров ферментов, необходимы идентификаторы белков в формате RefSeq (например, WP_000000000.0). Установление соответствия между локус-тегами генов и идентификаторами кодируемых ими белков является обязательным промежуточным этапом, обеспечивающим максимальное покрытие данных при объединении с UniProt и формировании итоговой обучающей выборки для моделей предсказания функций белков.

В качестве источника метаданных о геномных сборках прокариот были использованы файлы assembly_summary_bacteria.txt и assembly_summary_archaea.txt, предоставляемые NCBI RefSeq. Эти файлы содержат систематизированную информацию обо всех доступных сборках геномов бактерий и архей, включая таксономический идентификатор организма, уровень сборки (полный геном, скаффолды, контиги), категорию в RefSeq (референсный геном, репрезентативный геном), статус версии и пути File Transfer Protocol (FTP-пути) для скачивания аннотационных файлов.

Из объединённого набора сборок бактерий и архей были отобраны только те записи, таксономические идентификаторы которых присутствовали в списке организмов, полученном на предыдущем этапе предобработки данных ODB. Данная фильтрация позволила исключить сборки организмов, не представленных в наборе оперонов, и тем самым сократить объём последующих вычислений. Дополнительно из набора были удалены метагеномные сборки и записи, исключённые из RefSeq, чтобы обеспечить использование только качественных и таксономически однозначных геномных данных.

Для каждого таксономического идентификатора в базе NCBI может быть представлено несколько геномных сборок различного качества и полноты. Чтобы избежать дублирования данных и обеспечить использование наиболее полных и достоверных аннотаций, для каждого организма была выбрана единственная наилучшая сборка. Выбор осуществлялся по иерархии приоритетов: в первую очередь выбирался референсный геном; при его отсутствии – репрезентативный геном; далее полный геном; при отсутствии полного генома предпочтение отдавалось сборке с актуальным статусом версии (latest). Такой подход гарантировал, что для каждого организма использовалась сборка с максимально полной и актуальной аннотацией генов.

На основе FTP-путей выбранных сборок были автоматически сформированы URL-адреса для скачивания файлов аннотации в формате GFF (General Feature Format) [40]. Файлы GFF представляют собой стандартный табулированный формат аннотации геномных последовательностей, содержащий координаты генов, тип геномного элемента (кодирующая последовательность, рРНК, тРНК и др.), а также атрибуты, включая локус-тег и идентификатор белка RefSeq. Итоговая таблица, включающая таксономический идентификатор, код сборки и URL-адрес GFF-файла, была сохранена в формате Parquet для последующего использования.

2.3 Разбор GFF-файлов и построение таблицы соответствия

Для установления соответствия между локус-тегами генов из набора ODB и идентификаторами белков RefSeq было реализовано извлечение данных из загруженных GFF-файлов. Для каждого организма, представленного в отфильтрованном наборе сборок, выполнялись следующие операции: определялось множество локус-тегов генов данного организма из набора оперонов; загружался и распаковывался соответствующий GFF-файл в формате gzip; из файла извлекались записи кодирующих последовательностей, содержащие атрибуты locus_tag и protein_id [41]. Для каждой записи с кодирующими последовательностями, локус-тег которой совпадал с одним из генов в наборе оперонов, формировалась пара «локус-тег – идентификатор белка RefSeq».

Такой подход позволял связать опероннную принадлежность гена из ODB с конкретным белковым продуктом из RefSeq, что является ключевым шагом для последующего объединения с данными UniProt. Записи с кодирующими последовательностями, не содержащие атрибута локус-тега или идентификатора белка (например, псевдогены или некодирующие области), пропускались. Обработка GFF-файлов

проводилась последовательно для всех организмов; в случае ошибки загрузки или извлечения отдельного файла обработка продолжалась для оставшихся организмов с фиксацией информации об ошибке.

Результатом данного этапа являлась таблица соответствия, содержащая три столбца: таксономический идентификатор организма (taxid), локус-тег гена (locus_tag) и идентификатор белка RefSeq (refseq_protein_id). Эта таблица обеспечивает связующее звено между данными об оперонной организации генов и аминокислотными последовательностями белков, что позволяет на следующем этапе выполнить объединение с базой данных UniProt, содержащей EC-номера ферментов и полные последовательности белков – основные входные данные для нейросетевых моделей предсказания функций.

2.4 Загрузка и фильтрация данных UniProt

Для получения аминокислотных последовательностей белков и аннотаций их ферментативных функций использовалась база данных UniProtKB, доступ к которой осуществлялся через REpresentational State Transfer Application Programming Interface (REST) Application Programming Interface (API) сервиса UniProt [3]. Запросы к API формировались на основе списка таксономических идентификаторов организмов, полученного на этапе предобработки данных ODB. Для каждого запроса извлекались следующие поля: идентификатор записи UniProt (accession), таксономический идентификатор организма (organism_id), названия генов (gene_names), перекрёстные ссылки на базу данных NCBI RefSeq (xref_refseq), номера EC ферментов (ec) и полная аминокислотная последовательность (sequence).

Загрузка данных выполнялась пакетами по 15 таксономических идентификаторов в одном запросе, что обеспечивало баланс между скоростью загрузки и устойчивостью к ошибкам сети. Между запросами выдерживалась пауза в 1,5 секунды для соблюдения ограничений на частоту обращений к API. Для обеспечения отказоустойчивости при обработке большого числа организмов была реализована система контрольных точек: после каждого успешно загруженного пакета номер текущего пакета сохранялся в файл, что позволяло возобновить загрузку с последней успешной позиции в случае разрыва соединения или иного сбоя.

Полученный файл содержал записи для всех белков, принадлежащих организмам из набора оперонов, однако не все записи были пригодны для дальнейшего анализа. Фильтрация выполнялась в несколько этапов. На первом этапе из набора исключались

записи, не содержащие перекрёстной ссылки на базу данных RefSeq, поскольку эта ссылка необходима для установления соответствия между записями UniProt и идентификаторами белков, полученными на этапе обработки GFF-файлов (раздел 2.4). На втором этапе выполнялась нормализация поля RefSeq: записи, содержащие множественные идентификаторы RefSeq через разделитель, развёртывались в отдельные строки. Из полученного набора сохранялись только записи с идентификаторами формата WP_ (white paper protein), соответствующими неизбыточным белкам NCBI RefSeq; записи с идентификаторами других форматов исключались.

На третьем этапе выполнялась нормализация поля EC-номеров. Поле EC number в записях UniProt может содержать один или несколько EC-номеров через разделитель. Значения этого поля были преобразованы в списки строк; записи с полностью отсутствующими EC-номерами были исключены из набора, поскольку отсутствие функциональной аннотации делает их непригодными для задачи предсказания EC-номеров. Заключительным этапом являлось удаление дубликатов по комбинации полей Entry (идентификатор UniProt) и RefSeq (идентификатор белка RefSeq).

Итоговый набор данных был сохранён в формате Apache Parquet и использовался на следующем этапе для объединения с данными об оперонной организации генов (раздел 2.6).

2.5 Итоговое объединение и характеристики объединённого набора

В результате последовательного объединения данных из трёх источников (ODB, NCBI RefSeq и UniProtKB) был сформирован объединённый набор данных, содержащий 2363899 записей. Каждая запись связывает конкретный белок с его оперонной принадлежностью, аминокислотной последовательностью и аннотированным EC-номером фермента.

Покрытие данных на каждом этапе объединения представлено в таблице 1. На этапе сопоставления генов оперонов с белками RefSeq покрытие составило 84,0 %, что объясняется различиями в версиях геномных аннотаций между ODB и актуальными GFF-файлами NCBI, а также отсутствием кодирующих последовательностей для некоторых локус-тегов. На этапе объединения с UniProt покрытие составило 99,4 %, что свидетельствует о высокой полноте перекрёстных ссылок между RefSeq и UniProt. Общее покрытие от исходного набора оперонов составило 78,6 %. Потеря 147552 записей при финальной фильтрации обусловлена исключением оперонов, содержащих менее двух уникальных белков после всех этапов объединения.

Таблица 1 – покрытие данных на этапах объединения.

Параметр	Значение
Исходных записей оперонов (ODB)	3007902
После объединения ODB с RefSeq	2527381 (84,0 %)
После объединения RefSeq с UniProt	2511453 (99,4 %)
После дедупликации	2511451
Валидных оперонов после устранения дублирования	117594
Записей исключено при фильтрации по размеру оперона	147552
Записей после фильтрации	2363899 (78,6 %)

Количественные характеристики объединённого набора данных представлены в таблице 2. После устранения дублирования оперонов, при котором каждый белок в пределах одного организма относится к одному оперону, набор включает 159640 уникальных белков, принадлежащих к 117594 оперонам из 2095 организмов. Общее число уникальных EC-номеров ферментов составило 1714. Подавляющее большинство записей содержат белки с одним EC-номером, тогда как меньшая часть соответствует многофункциональным белкам, аннотированным двумя и более EC-номерами.

Медианная длина аминокислотных последовательностей составила 334 остатка при средней длине 382 остатка, что характерно для типичных прокариотических белков. Минимальная длина последовательности составила 29 аминокислотных остатков, максимальная – 1654. Медианный размер оперона равен 3 белкам.

Таблица 2 – количественные характеристики объединённого набора данных

Параметр	Значение
Уникальных белков (Entry)	159640
Уникальных оперонов	117594
Уникальных организмов	2095
Уникальных EC-номеров	1714
Длина последовательности (медиана)	334 а.к.
Длина последовательности (среднее)	382 а.к.
Длина последовательности (мин. / макс.)	29 / 1654 а.к.
Размер оперона (медиана)	3 белка

Полученный набор данных представляет собой промежуточный результат формирования обучающей выборки. На следующем этапе данный набор будет обогащён

информацией о принадлежности белков к метаболическим путям, что позволит дополнительно усилить функциональный контекст при предсказании EC-номеров и применении оперонного фильтра.

2.6 Подготовка оценочного набора

Для корректной оценки моделей предсказания функций белков необходимо обеспечить строгое разделение обучающих и тестовых данных, исключив из оценочного набора белки, присутствующие в обучающих выборках используемых моделей.

В качестве исходного набора данных использовался объединённый набор, сформированный на предыдущих этапах и содержащий 159640 уникальных белков, распределённых по 117594 оперонам после устранения дублирования. Для устранения утечки данных были загружены обучающие выборки обеих моделей: 153595 последовательностей ProteInfer из TFRecord-файла [42] и 191460 последовательностей CLEAN из таблицы кластеризации. Пересечение между обучающими выборками составило 62755 последовательностей, а объединённое множество включало 282300 уникальных последовательностей. Все записи, последовательности которых совпадали с последовательностями из объединённой обучающей выборки, были удалены, в результате чего число уникальных белков сократилось до 108267, а число оперонов до 83244. После исключения оперонов, содержащих менее двух уникальных белков, в наборе осталось 76302 уникальных белка и 48214 оперонов.

2.7 Обогащение метаболическими путями KEGG

Для реализации оперонного фильтра, основанного на функциональной согласованности белков оперона по метаболическим путям, оценочный набор был обогащён информацией о принадлежности ферментов к путям базы данных KEGG [2]. Обогащение выполнялось на основе файла pathid_ec.tsv, содержащего отображения между EC-номерами ферментов и идентификаторами метаболических путей. Файл включал 6519 связей, покрывая 3748 уникальных EC-номеров и 167 уникальных путей KEGG. Покрытие EC-номеров оценочного набора составило 70,4 %.

После обогащения из набора были исключены опероны, в которых менее двух белков имеют аннотацию метаболического пути. Для таких оперонов применение оперонного фильтра невозможно из-за отсутствия достаточного числа соседей с известным функциональным контекстом. В результате фильтрации было исключено 17237 таких

оперонов, что составило 35,7 % от общего числа оперонов. Итоговый оценочный набор данных содержал 51890 уникальных белков и 30977 оперонов.

2.8 Стратификация оценочного набора по уровню гомологии

Для дифференцированной оценки моделей в зависимости от степени сходства тестовых белков с обучающими данными была проведена стратификация оценочного набора с помощью инструмента Many-against-Many sequence searching (MMSeqs2) [43], обеспечивающего быстрый поиск сходных последовательностей в больших наборах данных. Для каждой последовательности оценочного набора вычислялся максимальный процент идентичности к ближайшей последовательности объединённой обучающей выборки, на основании которого белок относился к одной из четырёх страт: HIGH (идентичность > 70 %), MEDIUM (40 % ≤ идентичность ≤ 70 %), LOW (идентичность < 40 %) и NONE (сходных последовательностей не найдено). В работе использовалась версия MMseqs2 18.8cc5c. Поиск проводился с чувствительностью s = 7,5 и значением e-value = 10. Пороги соответствуют биологическим границам интерпретации идентичности последовательностей: 40 % нижняя граница идентичности по Rost [44]; 70 % рекомендованная граница высокой идентичности в ProteInfer [34]. Аналогичные пороги используются в CLEAN [30].

2.9 Анализ степени пересечения метаболических путей KEGG в оперонах

Применимость предложенного оперонного фильтра принципиально опирается на гипотезу о высокой степени пересечения метаболических путей KEGG у генов одного оперона (функциональной однородности оперонов): предполагается, что белки, входящие в состав одного оперона, преимущественно участвуют в общих метаболических путях. Для проверки данной гипотезы на материале исходной выборки оперонов был выполнен количественный анализ функциональной однородности по аннотациям метаболических путей KEGG.

Анализ проводился на наборе данных, полученном в результате объединения оперонной разметки ODB с аннотациями UniProt и геномными координатами RefSeq (раздел 2.7). Перед расчётом показателя однородности применялись две процедуры очистки выборки. Во-первых, выполнялась дедупликация белков, попадающих в несколько оперонов одновременно: такие белки находятся в граничных позициях соседствующих оперонов и могут быть отнесены сразу к нескольким группам по результатам процедуры

формирования набора. Для каждого такого белка сохранялось его сопоставление с тем опероном, который имеет наибольший размер; в остальных оперонах данный белок исключался. Во-вторых, после дедупликации опероны размером в один белок, возникшие как следствие отнесения граничного белка к другому оперону, исключались из дальнейшего анализа, поскольку для оперона из одного белка показатель функциональной однородности не определён. Из набора данных, прошедшего две описанные процедуры, выделялось подмножество оперонов, в которых хотя бы у одного белка имелась аннотация принадлежности к метаболическому пути KEGG. Это подмножество составило 88942 оперона из 117594; оставшиеся 28652 оперона без единого аннотированного белка исключались из расчёта как принципиально неинформативные для проверки гипотезы. Также из расчёта были исключены три глобальные KEGG-карты – «Метаболические пути» (rn01100), «Биосинтез вторичных метаболитов» (rn01110) и «Микробный метаболизм в различных средах» (rn01120), – которые по определению пересекаются с подавляющим большинством специфических путей и потому систематически завышают оценку однородности до значений, не отражающих биологическую согласованность.

Для каждого оперона O вычислялся количественный показатель функциональной однородности по формуле 2.1:

$$\mathrm{score}(O) = \frac{1}{K_o} \times \sum_{p=1}^{K_o} \frac{g_p}{G_o} \qquad (2.1)$$

, где K_o – число различных метаболических путей, в которых участвует хотя бы один белок оперона O; g_p – число белков оперона O, аннотированных как участвующие в пути p; G_o – число белков оперона O, имеющих аннотации метаболических путей KEGG.

По смыслу формулы score(O) представляет собой среднее по всем представленным в опероне путям значение доли аннотированных белков оперона, участвующих в каждом из этих путей. Значение score(O) = 1,0 достигается в случае, когда все аннотированные белки оперона участвуют в одном и том же наборе путей; меньшие значения отражают расщеплённость аннотаций между несколькими путями.

Для визуализации полученные значения score(O) агрегировались в виде гистограммы с десятью равными бинами на интервале [0; 1]. Дополнительно вычислялись доли оперонов с показателем однородности выше пороговых значений 0,3, 0,5, 0,7, 0,8, 0,9 и точно равным 1,0 – для количественной характеристики распределения. Результаты анализа приведены в разделе 3.1.

2.10 Оперонный фильтр

Оперонный фильтр представляет собой механизм постобработки, использующий функциональный контекст соседних генов в опероне для верификации предсказаний EC-номеров, полученных от нейросетевых моделей. Основная идея фильтра основана на биологической закономерности, где результаты были согласованы с гипотезой в разделе 3.1: гены, входящие в состав одного оперона, с высокой вероятностью участвуют в общих или функционально связанных метаболических путях. Следовательно, предсказанная функция фермента должна быть согласована с функциями его соседей по оперону.

Для каждого белка фильтр вычисляет оценку согласованности предсказания с оперонным контекстом. Пусть для белка получено предсказание EC-номера, которому соответствует множество метаболических путей KEGG. Для данного белка определяется множество путей, ассоциированных с аннотированными EC-номерами его соседей в том же опероне. Под соседями понимаются все белки, входящие в тот же оперон, за исключением рассматриваемого. Оценка определяется следующим образом. Значение 1 присваивается, если выполняется хотя бы одно из двух условий: множество метаболических путей, соответствующих предсказанному EC-номеру, пересекается с множеством путей соседей по оперону; предсказанный EC-номер совпадает с EC-номером хотя бы одного из соседних белков. В противном случае оценка принимает значение 0, что интерпретируется как несогласованность предсказания с оперонным контекстом.

Для белков, у которых отсутствует аннотированный функциональный контекст (то есть ни один из соседей не имеет ассоциированных метаболических путей), используется стратегия пропуска: таким предсказаниям присваивается специальное значение NO_CONTEXT, и они не подвергаются фильтрации. Если предсказанный EC-номер имеет хотя бы один общий метаболический путь с EC-номерами соседних белков по оперону или совпадает с EC-номером хотя бы одного из них, предсказание принимается; в противном случае отклоняется.

Следует отметить, что в данной работе фильтр применяется к предсказаниям, предварительно отфильтрованным по порогу уверенности, обозначаемое далее τ, что позволяет отделить влияние уверенности модели от влияния оперонного контекста. Таким образом, оперонный фильтр выступает независимым механизмом верификации предсказаний, основанным исключительно на биологическом контексте.

В итоговом оценочном наборе все белки имели не менее двух соседей с аннотациями метаболических путей, что обеспечило практически полную применимость фильтра и минимальную необходимость обращения к механизму пропуска.

2.11 Метрики оценки качества предсказаний

Для оценки качества предсказаний EC-номеров и эффективности оперонного фильтра использовались три основные метрики: F1 macro, accuracy и coverage (покрытие).

F1 macro вычисляется как среднее арифметическое значений F1-меры по всем классам EC-номеров, присутствующим в оценочном наборе. Для каждого класса с значение F1 рассчитывается как гармоническое среднее precision и recall по формуле 2.3:

$$\mathrm{Precision(c)} = \frac{TP(\mathrm{c})}{TP(\mathrm{c})+FP(\mathrm{c})} \tag{2.2}$$

$$\mathrm{Recall(c)} = \frac{TP(\mathrm{c})}{TP(\mathrm{c})+FN(\mathrm{c})} \tag{2.3}$$

$$\mathrm{F1(c)} = \frac{2 \times \mathrm{Precision(c)} \times \mathrm{Recall(c)}}{(\mathrm{Precision(c)} + \mathrm{Recall(c)})} \tag{2.4}$$

, где Precision(c) и Recall(c), в которых TP, FP; FN относятся к классу (c): TP(c) – число белков, корректно отнесённых к классу c; FP(c) – число белков, ошибочно отнесённых к классу c; FN(c) – число белков, с истинной аннотацией c, не предсказанной моделью как c. Итоговая метрика F1 macro представляет собой среднее арифметическое по всем классам и рассчитывалась по формуле 2.4:

$$\mathrm{F1\ macro} = \frac{1}{\mathrm{C}} \times \sum_{\mathrm{c}} \mathrm{F1} \tag{2.5}$$

, где C – число уникальных классов.

Выбор усреднения по типу macro обусловлен тем, что распределение EC-номеров в оценочном наборе является существенно несбалансированным [45]: отдельные EC-номера представлены тысячами белков, тогда как другие единичными. При использовании micro-усреднения вклад редких EC-номеров был бы подавлен частыми, что не позволило бы оценить способность моделей предсказывать функции малопредставленных ферментов.

Accuracy определяется как доля белков, для которых предсказанный EC-номер верхнего ранга (по L4) совпадает хотя бы с одним из аннотированных EC-номеров белка с истинными значениями и рассчитывается по формуле 2.5:

$$\mathrm{Accuracy} = \frac{\mathrm{N_correct}}{\mathrm{N_predicted}} \tag{2.6}$$

, где N_correct – число белков с корректным предсказанием, N_predicted – число белков, для которых модель выдала предсказание

Покрытие определяется как доля белков оценочного набора, для которых модель выдала хотя бы одно предсказание с уверенностью выше заданного порога, рассчитывалось по формуле 2.6:

$$\text{Покрытие} = \frac{\text{N_predicted}}{\text{N_total}} \quad (2.7)$$

, где N_total – общее число белков в оценочном наборе

Данная метрика характеризует охват модели и является важным дополнением к accuracy, поскольку модель может повышать точность за счёт отказа от предсказаний в неуверенных случаях.

Помимо оценки на уровне полных четырёхуровневых EC-номеров (L4), метрики рассчитывались также на уровнях L1 (класс фермента, первая цифра EC-номера), L2 (подкласс) и L3 (подподкласс). На каждом уровне предсказанные и истинные EC-номера обрезались до соответствующего уровня EC, после чего сравнивались стандартными метриками. Такая многоуровневая оценка позволяет выявить, на каком уровне детализации классификации модель и фильтр обеспечивают наибольший вклад.

2.12 Выбор оптимального порога уверенности

Обе рассматриваемые модели, ProteInfer и CLEAN, возвращают для каждого белка не только наиболее вероятный EC-номер, но и численную оценку, отражающую степень уверенности модели в этом предсказании. Для практического применения требуется определить пороговое значение уверенности, ниже которого предсказание отвергается как недостаточно уверенное. Выбор порога представляет собой компромисс между точностью предсказаний, которая растёт с повышением порога за счёт отсечения шумовых сигналов, и охватом, который при этом снижается из-за отказа от предсказаний с низкой уверенностью.

В работе для каждой из рассматриваемых моделей выполнялось сквозное сканирование по дискретному множеству значений порога в диапазоне от 0,00 до 0,95 со значениями 0,00, 0,30, 0,50, 0,60, 0,70, 0,80, 0,90 и 0,95. Нижняя граница диапазона определялась минимальным значением уверенности в файлах предсказаний модели ProteInfer; верхняя граница 0,95 выбрана как разумный предел перед насыщением, при котором покрытие становится критически малой.

На каждом значении порога вычислялись две группы метрик: базовые метрики модели без постобработки и метрики после применения оперонного фильтра, описанного в разделе 2.10. В качестве критерия выбора оптимального порога использовалось

максимальное значение F1 macro после применения оперонного фильтра. Данный критерий напрямую соответствует цели работы, а именно повышению точности предсказания EC-номеров за счёт оперонного функционального контекста и в отличие от критерия точки перелома базовой F1 объективен и не требует субъективной оценки величины существенного прироста. Тот же критерий применялся независимо к обеим моделям. Полученные оптимальные пороги и соответствующие им значения метрик представлены в разделе 3.2.

3 ОБСУЖДЕНИЕ РЕЗУЛЬТАТОВ

3.1 Характеристики оценочного набора и функциональная однородность оперонов

В результате процедур, описанных в разделах 2.6-2.8, был сформирован оценочный набор данных, содержащий 51890 уникальных белков и 30977 оперонов. Стратификация белков по уровню гомологии к обучающей выборке (раздел 2.8) обеспечила 9850921 совпадение для 51849 уникальных запросов. Распределение белков по стратам приведено в таблице 3.

Таблица 3 – распределение белков оценочного набора по стратам гомологии.

Страта	Критерий идентичности	Число белков (%)
HIGH	> 70 %	25176 (48,5 %)
MEDIUM	40 – 70 %	24574 (47,4 %)
LOW	< 40 %	2099 (4,0 %)
NONE	нет совпадений	41 (0,1 %)

Доминирование страт HIGH (48,5 %) и MEDIUM (47,4 %) отражает общую высокую степень охвата прокариотических белков в обучающих выборках используемых моделей, тогда как страта LOW (4,0 %), несмотря на свою малочисленность, представляет наибольший интерес для оценки, поскольку именно на ней проявляется способность моделей обобщать на низкогомологичные последовательности.

Для проверки гипотезы о функциональной однородности оперонов, лежащей в основе предложенного оперонного фильтра, был выполнен анализ распределения показателя однородности score(O) на материале исходной выборки 117594 оперонов по процедуре, описанной в разделе 2.9 Полученное распределение оценок представлено на рисунке 3.

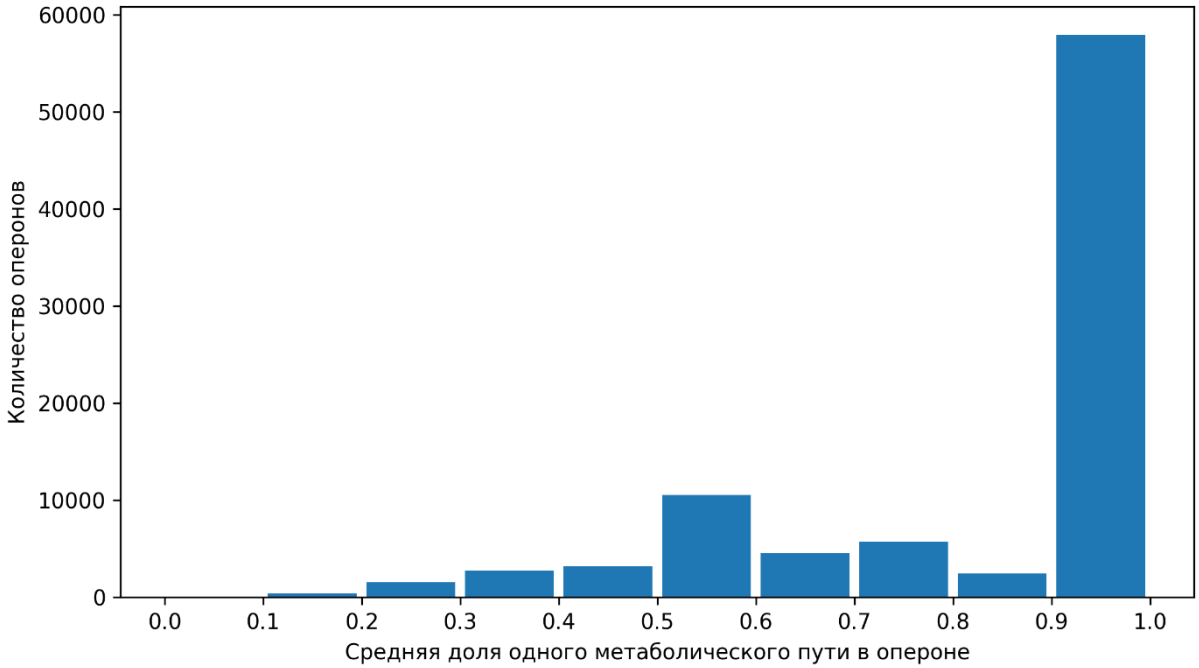


Рисунок 3 – распределение функциональной однородности оперонов по метаболическим путям KEGG (n = 117594 оперонов с известными аннотациями путей).

Распределение, представленное на рисунке 3, демонстрирует выраженную асимметрию в сторону высоких значений однородности. Количественные характеристики распределения приведены ниже: 70,8 % оперонов (88942 оперонов из 117594) имеют показатель однородности ≥ 0,9, что соответствует ситуации, когда практически все белки оперона относятся к одному и тому же метаболическому пути; для 70,7 % оперонов (57358) показатель равен 1,0, то есть все белки оперона участвуют в одном и том же наборе путей; Доля оперонов с низкой однородностью (< 0,3) невелика и составляет около 2.1%.

Полученные результаты эмпирически подтверждают биологическую гипотезу, лежащую в основе оперонного фильтра: подавляющее большинство оперонов в исследованных микроорганизмах демонстрируют высокую функциональную согласованность входящих в них белков по метаболическим путям [46]. Следовательно, использование пересечения путей предсказанного EC-номера с путями, аннотированными у соседей по оперону, является методологически обоснованным критерием для верификации предсказаний нейросетевых моделей [47]. Данный результат служит фундаментом для последующих экспериментов по оценке эффективности оперонного фильтра, представленных в разделах 3.2-3.4.

3.2 Базовая оценка моделей и выбор оптимального порога достоверности

Для оценки базовой производительности моделей ProteInfer и CLEAN, а также для объективного выбора порога уверенности предсказаний (τ) модели ProteInfer, использовалась процедура сквозного сканирования порогов с последующим применением критерия максимума F1 macro после оперонного фильтра – порога, при котором достигается максимальное значение F1 после функциональной верификации. Данный критерий напрямую соответствует цели работы – повышению точности предсказания EC-номеров через оперонный контекст. Для модели CLEAN порог τ выбирался по тому же критерию; сквозное сканирование её порогов приведено для сопоставления моделей и обоснования выбора ProteInfer как основной.

Для каждой из моделей оценка проводилась на восьми значениях порога τ в диапазоне от 0,00 до 0,95. На каждом пороге вычислялись метрики F1 macro и покрытие как для предсказаний модели без фильтрации, так и после применения оперонного фильтра. Результаты сквозного сканирования для обеих моделей приведены в таблице 4.

Таблица 4 – сквозное сканирование порога τ для моделей ProteInfer и CLEAN.

Порог	**F1 без фильтра**	**F1 + фильтр**	**Покрытие без фильтра**	**Покрытие + фильтр**	**ΔF1**
ProteInfer					
0,00	0,6383	0,8432	0,944	0,757	–
0,30	0,7982	0,8747	0,892	0,725	+0,0315
0,50	0,7980	0,8791	0,888	0,723	+0,0043
0,70	0,8018	0,8807	0,885	0,720	+0,0014
0,90	0.8074	0,8870	0,879	0,716	+0,0063
0,95	0,8052	0,8854	0,876	0,714	−0,0016
CLEAN					
0,00	0,5601	0,8315	0,992	0,770	–
0,30	0,6896	0,8650	0,920	0,727	+0,0335
0,50	0,7057	0,8704	0,898	0,713	+0,0054
0,70	0,7180	0,8744	0,868	0,693	+0,0040
0,90	0,7364	0,8760	0,793	0,637	+0,0016
0,95	0,7484	0,8787	0,730	0,586	+0,0027

Для наглядной визуализации динамики метрик в зависимости от порога τ построены отдельные диаграммы для F1 macro и покрытие обеих моделей в базовой конфигурации (без применения оперонного фильтра). Зависимость покрытия и F1 macro от порога τ представлена на рисунке 4.

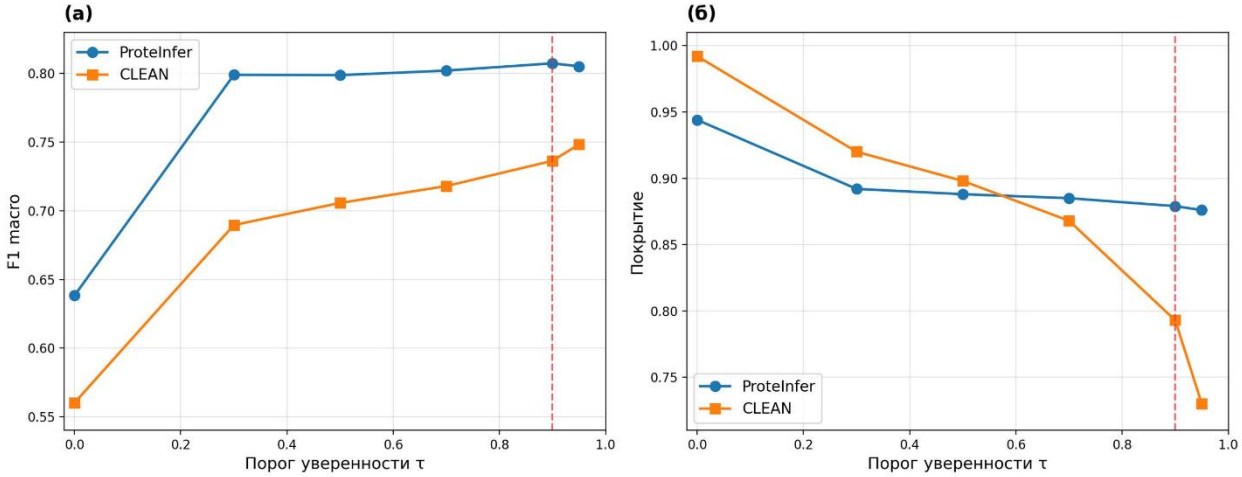


Рисунок 4 – зависимость метрик качества от порога уверенности τ для моделей ProteInfer и CLEAN без применения оперонного фильтра: (а) F1 macro, (б) coverage. Пунктирной линией отмечен выбранный порог τ* = 0,90 для Proteinfer.

Анализ таблицы показывает, что для модели ProteInfer F1 macro после применения фильтра монотонно растёт с повышением порога вплоть до τ = 0,90, где достигает максимума 0,8870, и снижается до 0,8854 при дальнейшем ужесточении до τ = 0,95. По критерию максимума F1 после верификации оптимальным порогом для ProteInfer выбирается τ = 0,90. Для модели CLEAN по тому же критерию максимум F1 macro после фильтра достигается при τ = 0,95 и составляет 0,8787 при покрытии 0,586. Несмотря на формальное наличие оптимума, ProteInfer при выбранном пороге устойчиво превосходит CLEAN по F1 macro (0,8870 против 0,8787) и обеспечивает существенно более высокое покрытие (0,716 против 0,586), что в сочетании с более высоким базовым качеством обосновывает выбор ProteInfer в качестве основной модели для последующих разделов.

При выбранном пороге τ = 0,90 базовые метрики модели ProteInfer на всём оценочном наборе (страта ALL) составили F1 macro 0.8074, accuracy 0,9666, покрытие 0,879; для модели CLEAN при выбранном по тому же критерию пороге τ = 0,95 – F1 macro 0,7484, accuracy 0,9376, покрытие 0,730. Распределение метрик по стратам гомологии представлено на рисунке 5.

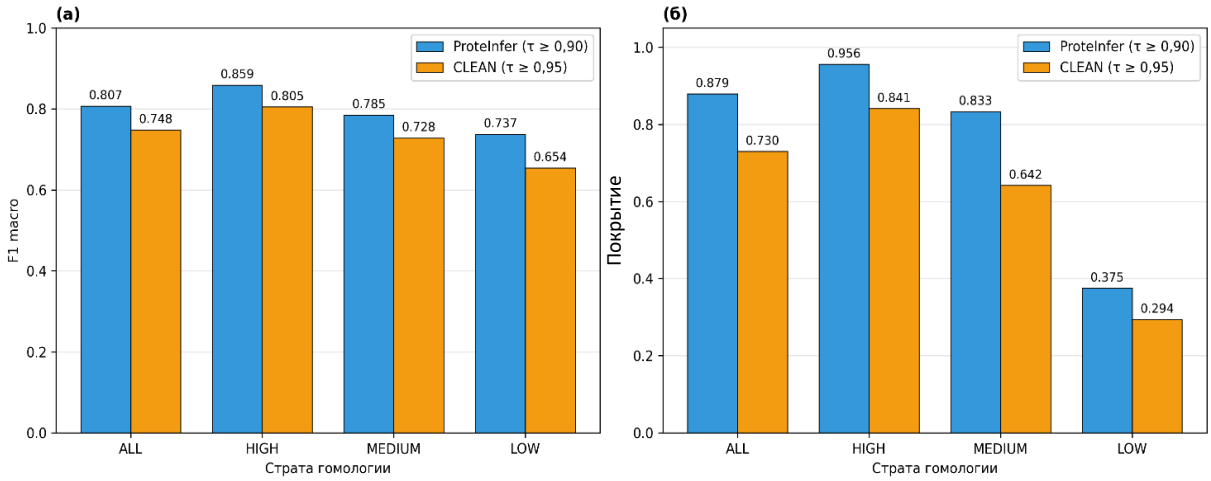


Рисунок 5 – сравнение моделей ProteInfer и CLEAN по F1 macro и покрытия в различных стратах гомологии к обучающей выборке. (а) Значения F1 macro; (б) значения покрытия.

Страты HIGH, MEDIUM и LOW соответствуют различным уровням сходства последовательностей с обучающей выборкой; страта NONE (41 белок, 0,1 % выборки) исключена из визуализации ввиду малого размера. Значения метрик приведены для конфигурации без применения оперонного фильтра при выбранных порогах уверенности: $\tau = 0{,}90$ для ProteInfer и $\tau = 0{,}95$ для CLEAN.

На рисунке 5 наблюдается устойчивое превосходство ProteInfer по F1 macro над CLEAN во всех стратах, что согласуется с архитектурными особенностями моделей: ProteInfer использует свёрточную нейросеть с остаточными связями, оптимизированную непосредственно на задаче классификации EC-номеров, тогда как CLEAN основана на контрастивном обучении векторных представлений и относит белок к EC-классу по расстоянию до центроидов обучающей выборки. При выбранных порогах, напротив, ProteInfer обеспечивает существенно более высокое покрытие (0,879 против 0,730 у CLEAN), сохраняя предсказания для большей доли белков оценочного набора. Сочетание превосходства по F1 macro и более высокого покрытие окончательно обосновывает выбор ProteInfer как основной модели.

Обе модели показывают ожидаемую деградацию F1 при переходе от страты HIGH к страте LOW, что отражает рост сложности задачи с уменьшением гомологии к обучающим данным. При этом деградация ProteInfer оказывается более плавной: F1 ProteInfer на LOW сохраняется на уровне 0,74, тогда как CLEAN снижает F1 до 0,65. Это делает ProteInfer более предпочтительной базовой моделью для дальнейших экспериментов, и именно её порог ($\tau = 0{,}90$) используется в последующих разделах как основной.

Дополнительно на рисунке 6 представлена Pareto-диаграмма зависимости F1 macro от покрытия для обеих моделей в базовой конфигурации и при применении фильтра. Выбранные оптимальные точки ProteInfer (PI) (τ = 0,90) и CLEAN (τ = 0,95) отмечены на диаграмме звёздами.

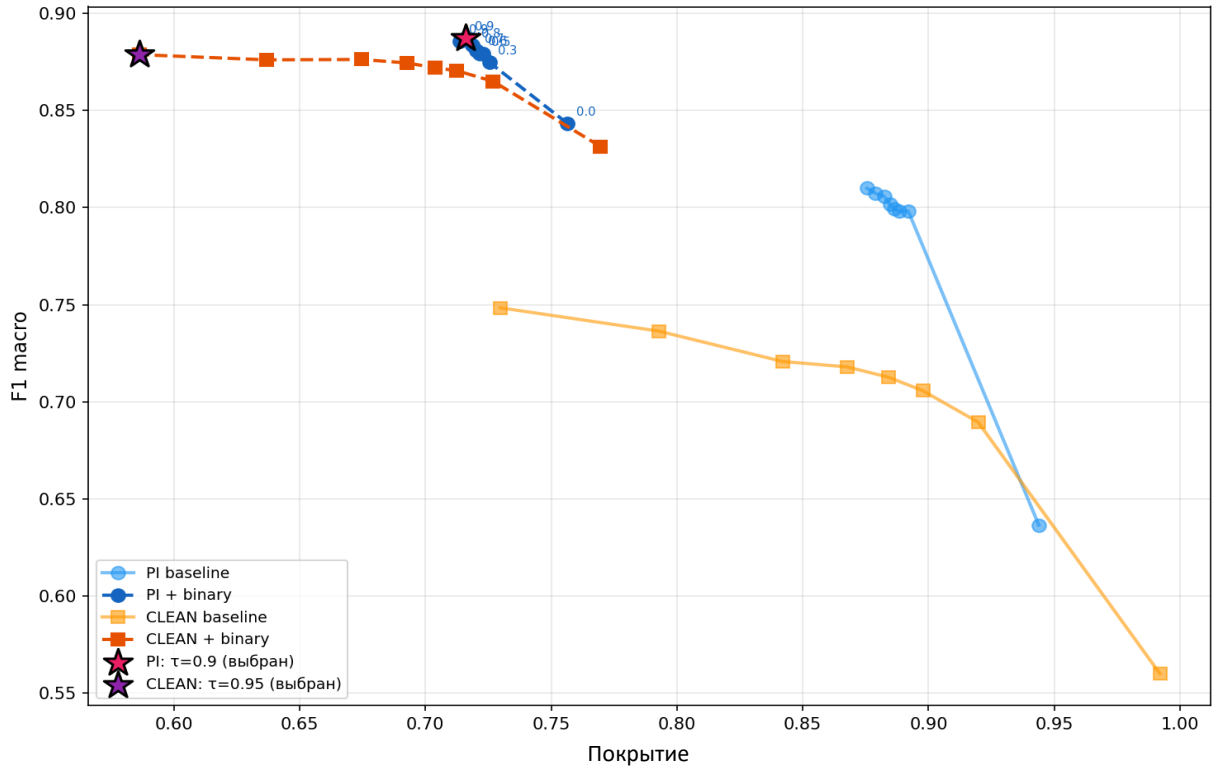


Рисунок 6 – pareto-диаграмма зависимости F1 macro от покрытия для моделей ProteInfer и CLEAN в конфигурациях baseline (без фильтра) и с применением оперонного фильтра.

Диаграмма на рисунке 6 наглядно демонстрирует, что применение оперонного фильтра сдвигает кривые обеих моделей вверх по оси F1 macro при умеренном снижении покрытия, причём этот эффект устойчив на всех рассмотренных порогах. Это позволяет предположить, что оперонный фильтр является универсальным механизмом улучшения качества предсказаний, не зависящим от выбора конкретной модели или порога уверенности – свойство, которое подробнее исследуется в последующих разделах.

3.3 Эффективность оперонного фильтра

Для количественной оценки эффективности разработанного в разделе 2.10 оперонного фильтра он был применён к предсказаниям ProteInfer при оптимальном пороге τ = 0,90 по каждой страте гомологии в отдельности. Результаты представлены в таблице 5.

Таблица 5 – влияние оперонного фильтра на метрики ProteInfer (τ = 0,90) по стратам гомологии.

Страта	F1 без фильтра	F1 + фильтр	Покрытие без фильтра	Покрытие + фильтр	ΔF1
HIGH	0,8595	0,9273	0,956	0,757	+0,0679
MEDIUM	0,7853	0,8443	0,833	0,705	+0,0590
LOW	0,7369	0,8608	0,375	0,276	+0,1239
ALL	0.8074	0,8870	0,879	0,716	+0,0797

Как видно из таблицы 5, оперонный фильтр обеспечивает неотрицательный прирост F1 macro во всех стратах гомологии, что подтверждает его безопасность как механизма постобработки: фильтр не ухудшает качество предсказаний ни в одной из рассмотренных категорий. Максимальный прирост F1 наблюдается на страте LOW (+0,1239), что представляет особый практический интерес: именно в условиях низкой гомологии к обучающим данным нейросетевые модели теряют точность, и именно в этих условиях оперонный контекст вносит наибольший корректирующий вклад. На страте HIGH прирост также существенен (+0,0679), однако объясняется он в первую очередь большим числом белков, на которых фильтр эффективно отсеивает редкие ошибочные предсказания. На страте MEDIUM эффект несколько меньше (+0,0590), но по-прежнему значим. Снижение покрытия составляет 10–20 процентных пунктов, что является допустимой платой за существенное повышение точности.

Для проверки устойчивости эффекта на различных уровнях детализации EC-классификации была выполнена оценка F1 macro на четырёх уровнях иерархии EC-номеров: L1 (класс), L2 (подкласс), L3 (подподкласс) и L4 (полный EC-номер). Результаты представлены в таблице 6 отдельно для каждой страты гомологии и для обеих моделей в базовых конфигурациях и с применением фильтра.

Таблица 6 – значения F1 macro на различных уровнях детализации EC-иерархии (L1–L4) для моделей ProteInfer (PI) и CLEAN по стратам гомологии.

Страта	Конфигурация	L1	L2	L3	L4
ALL	PI без фильтра	0,983	0,837	0,835	0,807
	PI + фильтр	0,992	0,921	0,897	0,887

Продолжение таблицы 6

Продолжение таблицы 6

Страта	Конфигурация	L1	L2	L3	L4
	CLEAN + фильтр	0,966	0,912	0,897	0,879
HIGH	PI без фильтра	0,985	0,848	0,878	0,859
	PI + фильтр	0,995	0,939	0,936	0,927
	CLEAN без фильтра	0,972	0,912	0,866	0,805
	CLEAN + фильтр	0,978	0,915	0,905	0,910
MEDIUM	PI без фильтра	0,981	0,847	0,795	0,785
	PI + фильтр	0,989	0,930	0,882	0,844
	CLEAN без фильтра	0,949	0,911	0,788	0,728
	CLEAN + фильтр	0,942	0,929	0,918	0,842
LOW	PI без фильтра	0,968	0,894	0,871	0,737
	PI + фильтр	0,978	0,961	0,953	0,861
	CLEAN без фильтра	0,963	0,784	0,803	0,654
	CLEAN + фильтр	1,000	1,000	1,000	0,838

Анализ таблицы 6 показывает закономерное снижение F1 при переходе от более общего уровня L1 к наиболее детальному уровню L4, отражающее рост сложности задачи с увеличением числа различаемых классов. Оперонный фильтр обеспечивает прирост F1 на всех уровнях иерархии для обеих моделей и во всех стратах, что свидетельствует об универсальности эффекта и отсутствии его привязки к какому-либо конкретному уровню классификации. Особого внимания заслуживают результаты на страте LOW: для модели CLEAN на уровне L4 прирост F1 составляет +0,184 (с 0,654 до 0,838), а для ProteInfer на том же уровне – +0,124. Таким образом, относительный прирост F1 максимален именно на глубоких уровнях классификации и на низкогомологичных белках, где базовое качество моделей наиболее низкое, а ценность дополнительного контекстного сигнала наиболее высока.

Дополнительно была проведена оценка эффективности фильтра в разрезе первого уровня EC-иерархии (L1) по 7 основным классам ферментов. Результаты в разбивке по классам и моделям представлены в таблице 7.

Таблица 7 – влияние оперонного фильтра на F1 macro в разрезе классов EC-иерархии первого уровня.

Класс	Название класса	Модель	F1 без фильтра	F1 + фильтр	ΔF1
EC 1	Оксидоредуктазы	ProteInfer	0,711	0,825	+0,114
		CLEAN	0,547	0,823	+0,276
EC 2	Трансферазы	ProteInfer	0,814	0,867	+0,053
		CLEAN	0,712	0,843	+0,131
EC 3	Гидролазы	ProteInfer	0,788	0,875	+0,087
		CLEAN	0,597	0,713	+0,116
EC 4	Лиазы	ProteInfer	0,753	0,828	+0,075
		CLEAN	0,684	0,796	+0,112
EC 5	Изомеразы	ProteInfer	0,745	0,822	+0,077
		CLEAN	0,829	0,892	+0,063
EC 6	Лигазы	ProteInfer	0,875	0,957	+0,082
		CLEAN	0,772	0,985	+0,213
EC 7	Транслоказы	ProteInfer	0,333	1,000	+0,667
		CLEAN	0,375	1,000	+0,625

Примечание – значения F1 без фильта и F1 с фильтром рассчитаны отдельно для каждого EC-класса первого уровня по белкам оценочного набора, отнесённым к данному классу.

Анализ таблицы 7 показывает, что прирост F1 macro неотрицателен для всех классов EC-иерархии у обеих моделей, что подтверждает отсутствие систематически вредных эффектов фильтра. Для шести основных классов ферментов (EC 1–EC 6) прирост F1 для модели ProteInfer находится в диапазоне от +0,053 до +0,114, а для модели CLEAN – от +0,063 до +0,276. Относительно больший прирост у CLEAN объясняется её более низким базовым качеством на L4: у моделей, дальше отстоящих от потолка F1, выше потенциал для коррекции.

Особого внимания заслуживают результаты по классу EC 7 (транслоказы), где прирост F1 достигает +0,667 для ProteInfer и +0,625 для CLEAN при абсолютном значении F1 = 1,000 после фильтра. Столь высокие значения следует интерпретировать с осторожностью: класс транслоказ был введён в номенклатуру IUBMB относительно

недавно и представлен в оценочном наборе незначительным числом белков, поэтому даже единичные корректные предсказания после фильтрации дают высокие значения F1. Этот результат демонстрирует потенциал метода, но не может рассматриваться как статистически надёжная оценка.

В целом, полученные результаты подтверждают, что оперонный фильтр представляет собой универсальный и безопасный механизм повышения точности предсказаний EC-номеров, эффективность которого не зависит от страты гомологии, уровня EC-иерархии или конкретного класса ферментов.

3.4 Независимость уверенности модели и оперонного контекста

Результаты раздела 3.3 продемонстрировали, что оперонный фильтр обеспечивает устойчивый прирост F1 macro во всех стратах гомологии и на всех уровнях EC-иерархии. Естественным развитием этого наблюдения является вопрос о природе данного эффекта: какую именно информацию вносит оперонный контекст и является ли она независимой от сигнала, на котором основаны предсказания нейросетевой модели. Если бы оперонный фильтр отклонял исключительно те предсказания, в которых модель сама недостаточно уверена, его действие сводилось бы к повышению порога τ и не добавляло нового источника информации. Если же фильтр способен обнаруживать ошибки и среди предсказаний, прошедших высокий порог уверенности, это означает, что оперонный фильтр несёт самостоятельный, независимый от уверенности модели сигнал [48].

Для проверки был выполнен анализ ошибок модели ProteInfer на выбранном рабочем пороге $\tau = 0{,}90$ (раздел 3.2). Под ошибкой понималось предсказание, прошедшее порог уверенности, но не совпадающее с истинным EC-номером. Для каждой страты гомологии вычислены две величины: доля таких ошибок среди всех белков страты, прошедших порог, и доля ошибок, отклонённых оперонным фильтром (recall фильтра, рассматриваемого как классификатор ошибок модели). Результаты приведены в таблице 8.

Таблица 8 – доля ошибок ProteInfer на выбранном пороге $\tau = 0{,}90$ и эффективность их обнаружения оперонным фильтром по стратам гомологии.

Страта	Доля ошибок, % от белков страты	Доля ошибок, пойманных фильтром (recall)
HIGH	2,4	60,3 %
MEDIUM	4,2	51,4 %
LOW	12,3	88,0 %

Данные таблицы 8 представляют собой наиболее наглядное подтверждение независимости уверенности модели и оперонного фильтра. Во-первых, наблюдается ожидаемый рост доли ошибок модели с уменьшением гомологии к обучающим данным: от 2,4 % на страте HIGH до 12,3 % на страте LOW более чем пятикратный рост. Этот результат отражает известную проблему нейросетевых моделей: прохождение порога уверенности не гарантирует корректности предсказания, особенно для белков, отличающихся от обучающей выборки.

Во-вторых, и это является ключевым наблюдением, эффективность обнаружения таких ошибок оперонным фильтром растёт вместе с ростом их доли. На страте HIGH фильтр обнаруживает 60,3 % ошибок, на MEDIUM равном 51,4 %, а на страте LOW, где ошибок больше всего (88,0 %), то есть почти девять из каждых десяти ошибок модели. Практический смысл этого наблюдения состоит в следующем: оперонный фильтр наиболее эффективен именно там, где нейросетевая модель наиболее склонна к ошибкам, то есть на низкогомологичных белках. Это делает фильтр особенно ценным для задач аннотации плохо изученных микроорганизмов и метагеномных данных [49, 50], где большинство белков имеют низкое сходство с обучающими наборами.

Суммируя результаты раздела 3.4, следует выделить три принципиальных вывода. Во-первых, даже среди предсказаний, прошедших высокий порог уверенности $\tau = 0{,}90$, остаётся существенная доля ошибок модели – от 2,4 % на страте HIGH до 12,3 % на страте LOW, – что означает: высокая уверенность не является достаточным критерием корректности, особенно для белков, эволюционно удалённых от обучающей выборки. Во-вторых, оперонный фильтр способен обнаруживать значительную часть этих ошибок, причём эффективность обнаружения растёт пропорционально частоте ошибок и достигает максимума на страте LOW (88,0 % пойманных ошибок). В-третьих, факт обнаружения ошибок при высокой уверенности предсказаний модели (0,90) означает, что оперонный контекст несёт самостоятельный сигнал, ортогональный уверенности модели; этим и объясняется устойчивый прирост F1 macro, наблюдавшийся в разделе 3.3.

ВЫВОДЫ

В ходе настоящей работы был разработан и апробирован метод повышения точности предсказания EC-номеров прокариотических белков на основе оперонного функционального контекста, применяемого в качестве фильтра предсказаний моделей глубокого обучения. Основные результаты работы можно сформулировать в виде следующих положений.

1. Сформирован репрезентативный набор данных, объединяющий информацию из Operon DataBase, NCBI RefSeq, UniProtKB и KEGG. После исключения последовательностей обучающих выборок моделей и дублирования оперонов итоговый набор содержал 51890 уникальных белков и 30977 оперонов из 2095 микроорганизмов.

2. Анализ функциональной однородности оперонов показал, что более 70 % оперонов имеют средний показатель однородности ≥ 0,9 по метаболическим путям KEGG, что наглядно подтверждает биологическую обоснованность использования оперонного контекста в качестве источника функциональной информации.

3. Проведена сравнительная оценка моделей глубокого обучения ProteInfer и CLEAN; по критерию максимума F1 и покрытия. В качестве основной модели выбрана ProteInfer при пороге уверенности модели равному 0,90, превзошедший CLEAN по обеим метрикам при выбранных порогах.

4. Разработан и реализован оперонный фильтр, использующий пересечение метаболических путей KEGG, ассоциированных с предсказанным EC-номером, и путей соседних белков того же оперона. Применение фильтра к предсказаниям ProteInfer повысило F1 macro до 0,887 при умеренном снижении покрытия; прирост неотрицателен во всех стратах гомологии и на всех уровнях EC-иерархии, что подтверждает безопасность и универсальность метода.

5. Оценен вклад оперонного контекста по стратам гомологии: эффективность фильтра возрастает с уменьшением сходства тестовых белков с обучающей выборкой. Наибольший прирост F1 macro достигнут на страте LOW (+0,124), а среди ошибок модели на выбранном пороге фильтр обнаруживает 88 % на страте LOW против 60 % на HIGH, что показывает: оперонный контекст компенсирует ограничения моделей глубокого обучения на низкогомологичных белках.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

API – Application Programming Interface, программный интерфейс приложения

BLAST – Basic Local Alignment Search Tool

CAFA – Critical Assessment of Function Annotation

CLEAN – Contrastive Learning enabled Enzyme Annotation

EC – Enzyme Commission, классификационный номер фермента

ESM – Evolutionary Scale Modeling

GFF – General Feature Format

GFF – Gene Ontology

IUBMB – International Union of Biochemistry and Molecular Biology, Международный союз биохимии и молекулярной биологии

KEGG – Kyoto Encyclopedia of Genes and Genomes

MMseqs2– Many-against-Many sequence searching, version 2

NCBI – National Center for Biotechnology Information

NGS – Next-Generation Sequencing, секвенирование нового поколения

ODB – Operon DataBase

OMA – Orthologous MAtrix

PI – ProteInfer

RefSeq – Reference Sequence database (NCBI)

SVM – Support Vector Machine, метод опорных векторов

UniProtKB – UniProt Knowledgebase

СПИСОК ЦИТИРОВАННОЙ ЛИТЕРАТУРЫ

1. Schnoes A. M., Brown S. D., Dodevski I., Babbitt P. C. Annotation error in public databases: misannotation of molecular function in enzyme superfamilies // PLoS Computational Biology. 2009. Vol. 5, No. 12. P. 1–13. DOI: 10.1371/journal.pcbi.1000605.

2. Kanehisa M., Goto S. KEGG: Kyoto Encyclopedia of Genes and Genomes // Nucleic Acids Research. 2000. Vol. 28, No. 1. P. 27–30. DOI: 10.1093/nar/28.1.27.

3. The UniProt Consortium. UniProt: the Universal Protein Knowledgebase in 2023 // Nucleic Acids Research. 2023. Vol. 51, No. 1. P. 523–531. DOI: 10.1093/nar/gkac1052.

4. The UniProt Consortium. UniProt: the Universal Protein Knowledgebase in 2025 // Nucleic Acids Research. 2025. Vol. 53, No. 1. P. 609–617. DOI: 10.1093/nar/gkae1010.

5. Morgat A., Lombardot T., Coudert E., Axelsen K., Neto T. B., Gehant S., Bansal P., Bolleman J., Gasteiger E., de Castro E., Baratin D., Pozzato M., Xenarios I., Poux S., Redaschi N., Bridge A., The UniProt Consortium. Enzyme annotation in UniProtKB using Rhea // Bioinformatics. 2020. Vol. 36, No. 6. P. 1896–1901. DOI: 10.1093/bioinformatics/btz817.

6. Altschul S. F., Gish W., Miller W., Myers E. W., Lipman D. J. Basic local alignment search tool // Journal of Molecular Biology. 1990. Vol. 215, No. 3. P. 403–410. DOI: 10.1016/S0022-2836(05)80360-2.

7. Radivojac P., Clark W. T., Oron T. R. et al. A large-scale evaluation of computational protein function prediction // Nature Methods. 2013. Vol. 10, No. 3. P. 221–227. DOI: 10.1038/nmeth.2340.

8. Jacob F., Monod J. Genetic regulatory mechanisms in the synthesis of proteins // Journal of Molecular Biology. 1961. Vol. 3, No. 3. P. 318–356. DOI: 10.1016/S0022-2836(61)80072-7.

9. Lim H. N., Lee Y., Hussein R. Fundamental relationship between operon organization and gene expression // Proceedings of the National Academy of Sciences. 2011. Vol. 108, No. 26. P. 10626–10631. DOI: 10.1073/pnas.1105692108.

10. Salgado H., Gama-Castro S., Peralta-Gil M. et al. RegulonDB (version 5.0): Escherichia coli K-12 transcriptional regulatory network, operon organization, and growth conditions // Nucleic Acids Research. 2006. Vol. 34, No. 1. P. 394–397. DOI: 10.1093/nar/gkj156.

11. Vey G. Metagenomic Guilt by Association: An Operonic Perspective // PLoS ONE. 2013. Vol. 8, No. 8. P. 71484–71495. DOI: 10.1371/journal.pone.0071484.

12. Okuda S., Yoshizawa A. C. ODB: a database for operon organizations, 2011 update // Nucleic Acids Research. 2011. Vol. 39, No. 1. P. 552–555. DOI: 10.1093/nar/gkq1090.

13. Mao F., Dam P., Chou J., Olman V., Xu Y. DOOR: a database for prokaryotic operons // Nucleic Acids Research. 2009. Vol. 37, No. 1. P. 459–463. DOI: 10.1093/nar/gkn757.

14. Price M. N., Wetmore K. M., Waters R. J. et al. Mutant phenotypes for thousands of bacterial genes of unknown function // Nature. 2018. Vol. 557, No. 7706. P. 503–509. DOI: 10.1038/s41586-018-0124-z.

15. Nelson D. L., Cox M. M. Lehninger Principles of Biochemistry. 7th ed. New York: W. H. Freeman, 2017. 1158 p.

16. Jeong H., Tombor B., Albert R., Oltvai Z. N., Barabási A.-L. The large-scale organization of metabolic networks // Nature. 2000. Vol. 407, No. 6804. P. 651–654. DOI: 10.1038/35036627.

17. Fell D. A. Understanding the Control of Metabolism. London: Portland Press, 1997. 301 p.

18. Kulmanov M., Khan M. A., Hoehndorf R. DeepGO: predicting protein functions from sequence and interactions using a deep ontology-aware classifier // Bioinformatics. 2018. Vol. 34, No. 4. P. 660–668. DOI: 10.1093/bioinformatics/btx624.

19. Gligorijević V., Renfrew P. D., Kosciolek T. et al. Structure-based protein function prediction using graph convolutional networks // Nature Communications. 2021. Vol. 12. P. 3168-3182. DOI: 10.1038/s41467-021-23303-9.

20. Overbeek R., Fonstein M., D'Souza M., Pusch G. D., Maltsev N. The use of gene clusters to infer functional coupling // Proceedings of the National Academy of Sciences. 1999. Vol. 96, No. 6. P. 2896–2901. DOI: 10.1073/pnas.96.6.2896.

21. Huynen M., Snel B., Lathe W., Bork P. Predicting protein function by genomic context: quantitative evaluation and qualitative inferences // Genome Research. 2000. Vol. 10, No. 8. P. 1204–1210. DOI: 10.1101/gr.10.8.1204.

22. Webb E. C. Enzyme Nomenclature 1992: Recommendations of the Nomenclature Committee of the IUBMB on the Nomenclature and Classification of Enzymes. San Diego: Academic Press, 1992. 862 p.

23. Boyce S., Tipton K. F. Enzyme classification and nomenclature // Encyclopedia of Life Sciences. Chichester: John Wiley & Sons, 2001. DOI: 10.1038/npg.els.0000710.

24. Bairoch A. The ENZYME database in 2000 // Nucleic Acids Research. 2000. Vol. 28, No. 1. P. 304–305. DOI: 10.1093/nar/28.1.304.

25. Prešern U., Goličnik M. Enzyme Databases in the Era of Omics and Artificial Intelligence // International Journal of Molecular Sciences. 2023. Vol. 24, No. 23. P. 16918–16936. DOI: 10.3390/ijms242316918.

26. Li L., Stoeckert C. J. Jr., Roos D. S. OrthoMCL: identification of ortholog groups for eukaryotic genomes // Genome Research. 2003. Vol. 13, No. 9. P. 2178–2189. DOI: 10.1101/gr.1224503.

27. Altenhoff A. M., Warwick Vesztrocy A., Bernard C., Train C.-M., Nicheperovich A., Prieto Baños S., Julca I., Moi D., Nevers Y., Majidian S., Dessimoz C., Glover N. M. OMA orthology in 2024: improved prokaryote coverage, ancestral and extant GO enrichment, a revamped synteny viewer and more in the OMA Ecosystem // Nucleic Acids Research. 2024. Vol. 52, No. 1. P. 513–521. DOI: 10.1093/nar/gkad1020.

28. Cai C. Z., Han L. Y., Ji Z. L., Chen X., Chen Y. Z. SVM-Prot: web-based support vector machine software for functional classification of a protein from its primary sequence // Nucleic Acids Research. 2003. Vol. 31, No. 13. P. 3692–3697. DOI: 10.1093/nar/gkg600.

29. Dalkiran A., Rifaioglu A. S., Martin M. J., Cetin-Atalay R., Atalay V., Doğan T. ECPred: a tool for the prediction of the enzymatic functions of protein sequences based on the EC nomenclature // BMC Bioinformatics. 2018. Vol. 19. P. 334–347. DOI: 10.1186/s12859-018-2368-y.

30. Yu T., Cui H., Li J. C., Luo Y., Jiang G., Zhao H. Enzyme function prediction using contrastive learning // Science. 2023. Vol. 379, No. 6639. P. 1358–1363. DOI: 10.1126/science.adf2465.

31. Bileschi M. L., Belanger D., Bryant D. H. et al. Using deep learning to annotate the protein universe // Nature Biotechnology. 2022. Vol. 40, No. 6. P. 932–937. DOI: 10.1038/s41587-021-01179-w.

32. Ashburner M., Ball C. A., Blake J. A. et al. Gene Ontology: tool for the unification of biology // Nature Genetics. 2000. Vol. 25, No. 1. P. 25–29. DOI: 10.1038/75556.

33. Rives A., Meier J., Sercu T. et al. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences // Proceedings of the National Academy of Sciences. 2021. Vol. 118, No. 15. DOI: 10.1073/pnas.2016239118.

34. Sanderson T., Bileschi M. L., Belanger D., Colwell L. J. ProteInfer, deep neural networks for protein functional inference // eLife. 2023. Vol. 12. DOI: 10.7554/eLife.80942.

35. Yu F., Koltun V. Multi-scale context aggregation by dilated convolutions // Proceedings of the International Conference on Learning Representations (ICLR). 2016. DOI: 10.48550/arXiv.1511.07122.

36. He K., Zhang X., Ren S., Sun J. Deep residual learning for image recognition // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016. P. 770–778. DOI: 10.1109/CVPR.2016.90.

37. Khosla P., Teterwak P., Wang C. et al. Supervised contrastive learning // Advances in Neural Information Processing Systems. 2020. Vol. 33. P. 18661–18673. DOI: 10.48550/arXiv.2004.11362.

38. O'Leary N. A., Wright M. W., Brister J. R. et al. Reference sequence (RefSeq) database at NCBI: current status, taxonomic expansion, and functional annotation // Nucleic Acids Research. 2016. Vol. 44, No. 1. P. 733–745. DOI: 10.1093/nar/gkv1189.

39. McKinney W. Data structures for statistical computing in Python // Proceedings of the 9th Python in Science Conference. 2010. P. 56–61. DOI: 10.25080/Majora-92bf1922-00a.

40. Eilbeck K., Lewis S. E., Mungall C. J., Yandell M., Stein L., Durbin R., Ashburner M. The Sequence Ontology: a tool for the unification of genome annotations // Genome Biology. 2005. Vol. 6, No. 5. P. 44–56 DOI: 10.1186/gb-2005-6-5-r44.

41. Tatusova T., DiCuccio M., Badretdin A., Chetvernin V., Nawrocki E. P., Zaslavsky L., Lomsadze A., Pruitt K. D., Borodovsky M., Ostell J. NCBI prokaryotic genome annotation pipeline // Nucleic Acids Research. 2016. Vol. 44, No. 14. P. 6614–6624. DOI: 10.1093/nar/gkw569.

42. Abadi M., Barham P., Chen J., Chen Z., Davis A., Dean J., Devin M., Ghemawat S., Irving G., Isard M., Kudlur M., Levenberg J., Monga R., Moore S., Murray D. G., Steiner B., Tucker P., Vasudevan V., Warden P., Wicke M., Yu Y., Zheng X. TensorFlow: a system for large-scale machine learning // Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16). 2016. P. 265–283.DOI: 10.48550/arXiv.1605.08695

43. Steinegger M., Söding J. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets // Nature Biotechnology. 2017. Vol. 35, No. 11. P. 1026–1028. DOI: 10.1038/nbt.3988.

44. Rost B. Twilight zone of protein sequence alignments // Protein Engineering. 1999. Vol. 12, No. 2. P. 85–94. DOI: 10.1093/protein/12.2.85.

45. Sokolova M., Lapalme G. A systematic analysis of performance measures for classification tasks // Information Processing & Management. 2009. Vol. 45, No. 4. P. 427–437. DOI: 10.1016/j.ipm.2009.03.002.

46. Moreno-Hagelsieb G., Collado-Vides J. A powerful non-homology method for the prediction of operons // Bioinformatics. 2002. Vol. 18, No. 1. P. 329–336. DOI: 10.1093/bioinformatics.

47. Enault F., Suhre K., Claverie J.-M. Phydbac «Gene Function Predictor» // Nucleic Acids Research. 2005. Vol. 33, No. 2. P. 393–396. DOI: 10.1093/nar/gki497.

48. Pellegrini M., Marcotte E. M., Thompson M. J., Eisenberg D., Yeates T. O. Assigning protein functions by comparative genome analysis: protein phylogenetic profiles // Proceedings of the National Academy of Sciences. 1999. Vol. 96, No. 8. P. 4285–4288. DOI: 10.1073/pnas.96.8.4285.

49. Hugenholtz P., Tyson G. W. Microbiology: metagenomics // Nature. 2008. Vol. 455, No. 7212. P. 481–483. DOI: 10.1038/455481a.

50. Danczak R. E., Goldman A. E., Chu R. K. et al. Using metacommunity ecology to understand environmental metabolomes // Nature Communications. 2020. Vol. 11, No. 1. P. 6369–6385.

ПРИЛОЖЕНИЕ

Исходный код скриптов предобработки данных

В настоящем приложении приведён полный исходный код семи скриптов на языке Python, реализующих этапы построения оценочного наборов данных, описанные в главе 2 настоящей работы. Скрипты приведены в порядке исполнения, соответствующем последовательности этапов конвейера предобработки. Каждый скрипт сопровождён комментариями, поясняющими ключевые операции.

Приложение 1

```
"""
0PATHID_PREPROCESS.py

Удаление трёх супер-агрегированных KEGG-карт из pathid_ec.tsv:
    01100  Metabolic pathways
    01110  Biosynthesis of secondary metabolites
    01120  Microbial metabolism in diverse environments

Эти карты содержат EC-номера со всех конкретных путей сразу. При оперонном
анализе они дают ложно-высокую функциональную связность: любые два белка
с EC-функцией матчатся через rn01100, что искажает оценку однородности.

Идемпотентность:
- При первом запуске оригинал создает резервную копию в pathid_ec.raw.tsv.
- При повторных запусках чтение идёт из резервной копии, что позволяет менять
  правила фильтрации без ручного восстановления файла.

Вход:  pathid_ec.tsv (или pathid_ec.raw.tsv, если бэкап уже создан)
Выход: pathid_ec.tsv         (очищенный, перезапись)
       pathid_ec.raw.tsv     (резервная копия оригинала, создаётся один раз)
"""


import os
import re
import shutil
import pandas as pd



BASE_DIR    = "/mnt/storage/home/sykarmanov/proteinfer/preprocessing"
PATHID_PATH = os.path.join(BASE_DIR, "pathid_ec.tsv")
BACKUP_PATH = os.path.join(BASE_DIR, "pathid_ec.raw.tsv")


# Регулярное выражение покрывает разные форматы записи KEGG-карт:
# rn01100, map01100, ko01100, path01100, либо просто 01100.
OVERVIEW_RE = re.compile(
    r"^(?:rn|map|ko|path)?(?:01100|01110|01120)$",
```

```
    re.IGNORECASE
)



def main():
    # Если резервная копия существует — читаем из него, обеспечивая идемпотентность
    # повторных запусков при изменении правил фильтрации.
    if os.path.isfile(BACKUP_PATH):
        source_path = BACKUP_PATH
        print(f"  [INFO] Читаю оригинал из резервной копии: {BACKUP_PATH}")
    elif os.path.isfile(PATHID_PATH):
        source_path = PATHID_PATH
    else:
        raise FileNotFoundError(
            f"Не найден ни {PATHID_PATH}, ни {BACKUP_PATH}"
        )


    df = pd.read_csv(source_path, sep="\t")
    n_in = len(df)


    # Автоопределение колонки с идентификатором пути.
    if "Path" in df.columns:
        path_col = "Path"
    else:
        candidates = [c for c in df.columns
                      if c.lower() in ("path", "pathway", "path_id",
                                       "pathid", "kegg_id")]
        if not candidates:
            raise KeyError(
                f"Не найдена колонка с KEGG path ID. Колонки: "
                f"{list(df.columns)}. Ожидалась 'Path' или похожая."
            )
        path_col = candidates[0]
        print(f"  [INFO] Используется колонка '{path_col}'.")


    df[path_col] = df[path_col].astype(str).str.strip()
    mask_overview = df[path_col].apply(lambda x: bool(OVERVIEW_RE.match(x)))
    to_remove = df.loc[mask_overview, path_col].value_counts().sort_index()


    print("\n  ID, помеченные как глобальные карты (будут удалены):")
    if len(to_remove) == 0:
        print("    (не найдены в файле)")
    else:
        for path_id, cnt in to_remove.items():
            print(f"    {path_id:>15}: {cnt} строк")


    df_clean = df.loc[~mask_overview].copy()
    n_out       = len(df_clean)
    n_removed   = n_in - n_out
    n_paths_in  = df[path_col].nunique()
    n_paths_out = df_clean[path_col].nunique()


    # Создаём резервную копию оригинала только при первом запуске.
    if not os.path.isfile(BACKUP_PATH):
        shutil.copy2(PATHID_PATH, BACKUP_PATH)
        print(f"\n  Бэкап создан: {BACKUP_PATH}")
    else:
        print(f"\n  Бэкап уже существует, не перезаписываю: {BACKUP_PATH}")


    df_clean.to_csv(PATHID_PATH, sep="\t", index=False)


    print(f"\n=== Препроцессинг pathid_ec.tsv ===")
```

```
    print(f"Входных строк:         {n_in}")
    print(f"Выходных строк:        {n_out} ({100*n_out/n_in:.1f}% от входа)")
    print(f"Удалено строк:         {n_removed}")
    print(f"Уникальных Path до:    {n_paths_in}")
    print(f"Уникальных Path после:{n_paths_out} ({n_paths_in - n_paths_out} удалено)")
    print(f"Записано в:            {PATHID_PATH}")



if __name__ == "__main__":
    main()
```

Приложение 2

```
"""
1OPERON_RS.py

Разбор данных Operon DataBase (ODB) из дампа conserved_operon.download.txt.
Из каждой записи об опероне извлекаются: идентификатор оперона (coid),
таксономический идентификатор организма (org) и список генов оперона (op).

Затем список генов разворачивается в длинный формат «один ген — одна
строка», удаляются дубликаты и опероны, содержащие менее двух генов.

Вход:  conserved_operon.download.txt
Выход: operon_genes_RS.parquet
       taxids.txt — список уникальных taxon_id для последующих шагов
"""

import pandas as pd

path = "conserved_operon.download.txt"

# Загрузка ODB.
df = pd.read_csv(
    path,
    sep="\t",
    header=0,
    dtype="string",
    low_memory=False
)

# Переименование колонок ODB к рабочим именам конвейера:
# coid -> operon_id (идентификатор оперона)
# org  -> taxon_id  (организм)
# op   -> genes     (список генов оперона, разделитель — запятая)
df = (
    df
    .rename(columns={
        "coid": "operon_id",
        "org": "taxon_id",
        "op": "genes"
    })[
        ["taxon_id", "operon_id", "genes"]
    ]
)

# Разворачивание списка генов в длинный формат: одна строка на ген.
df["gene_id"] = df["genes"].str.split(",")
df = df.explode("gene_id").drop(columns="genes")

df["gene_id"] = df["gene_id"].str.strip()
df = df.dropna(subset=["gene_id", "operon_id", "taxon_id"])

df = df.drop_duplicates(subset=["operon_id", "gene_id", "taxon_id"])

print(df.head(10))
print("Shape:", df.shape)
print("Топ operon_id:")
print(df.operon_id.value_counts().head())

# Подсчёт размеров оперонов: число уникальных генов на оперон.
```

```
operon_sizes = df.groupby("operon_id")["gene_id"].nunique()
print("Всего operon_id:", len(operon_sizes))


# Фильтрация: оперон по определению содержит >=2 ко-транскрибируемых гена,
# поэтому записи из одного гена исключаются как заведомо некорректные.
valid_operons = operon_sizes[operon_sizes >= 2].index
df = df[df["operon_id"].isin(valid_operons)]
print("Оперонов >=2 генов:", len(valid_operons))


# Сохранение списка организмов для последующего отбора сборок RefSeq.
taxids = df["taxon_id"].unique()


with open("taxids.txt", "w") as f:
    for t in taxids:
        f.write(str(t) + "\n")


# Контрольный вывод сводных величин.
print("Оперонов из 1 гена:",
      (df.groupby("operon_id")["gene_id"].nunique() == 1).sum())


n_operons = df[["operon_id", "taxon_id"]].drop_duplicates().shape[0]
print("Уникальные опероны:", n_operons)
print(f"Количество уникальных таксономических ID: {len(taxids)}")


df.to_parquet("operon_genes_RS.parquet", index=False)
print(df.head())
print("Shape:", df.shape)
```

Приложение 3

```
"""
2PARS_REFSEQ_PROK.py

Отбор лучшей геномной сборки для каждого организма из assembly_summary
NCBI RefSeq, ограниченный таксонами, присутствующими в наборе оперонов.

Логика отбора (по приоритету): reference genome -> representative genome
-> Complete Genome -> latest version. Если ни одно условие не выполнено,
берётся первая сборка по умолчанию.

Вход:  assembly_summary_bacteria.txt, assembly_summary_archaea.txt
       taxids.txt
Выход: selected_assemblies_with_gff.parquet
"""

import pandas as pd



def load_assembly_summary(path):
    # Файлы assembly_summary имеют комментарий-метаданные в первой строке,
    # поэтому пропускаем её через skiprows.
    df = pd.read_csv(
        path,
        sep="\t",
        skiprows=1,
        low_memory=False
    )
    df.columns = df.columns.str.replace("#", "", regex=False)
    df["taxid"] = df["taxid"].astype(int)
    return df



bacteria = load_assembly_summary("assembly_summary_bacteria.txt")
archaea  = load_assembly_summary("assembly_summary_archaea.txt")
assembly_df = pd.concat([bacteria, archaea], ignore_index=True)

# Загрузка списка taxid из набора оперонов (для ограничения сборок).
taxids = set(
    pd.read_csv("taxids.txt", header=None, dtype=int)[0]
)

assembly_df = assembly_df[assembly_df["taxid"].isin(taxids)]

# Исключение метагеномов и других нестандартных сборок:
# в колонке excluded_from_refseq содержится причина исключения.
assembly_df = assembly_df[
    (assembly_df["excluded_from_refseq"].isna()) |
    (assembly_df["excluded_from_refseq"] == "na")
]



def select_best_assembly(df):
    """Возвращает наиболее качественную сборку из набора сборок одного taxid.

    Приоритеты соответствуют качеству аннотации в RefSeq: эталонные геномы
    проходят ручную курацию, репрезентативные — автоматический отбор,
    Complete Genome означает законченную сборку без пробелов.
```

```
    """
    ref = df[df["refseq_category"] == "reference genome"]
    if len(ref):
        return ref.iloc[0]

    rep = df[df["refseq_category"] == "representative genome"]
    if len(rep):
        return rep.iloc[0]

    complete = df[df["assembly_level"] == "Complete Genome"]
    if len(complete):
        return complete.iloc[0]

    latest = df[df["version_status"] == "latest"]
    if len(latest):
        return latest.iloc[0]

    return df.iloc[0]



selected = (
    assembly_df
    .groupby("taxid", group_keys=False)
    .apply(select_best_assembly)
    .reset_index(drop=True)
)



def build_gff_url(row):
    # FTP-пути в assembly_summary указаны до директории сборки;
    # имя GFF-файла внутри директории — <basename>_genomic.gff.gz.
    base = row["ftp_path"].split("/")[-1]
    return f"{row['ftp_path']}/{base}_genomic.gff.gz"



selected["gff_url"] = selected.apply(build_gff_url, axis=1)

selected[["taxid", "assembly_accession", "gff_url"]].to_parquet(
    "selected_assemblies_with_gff.parquet",
    index=False
)

print("Saved selected_assemblies_with_gff.parquet")
```

Приложение 4

```
"""
3GFF_PARSER.py

Скачивание и разбор GFF-файлов из NCBI RefSeq для каждой отобранной сборки.
Из каждой записи типа CDS извлекаются атрибуты locus_tag и protein_id;
сохраняются только те locus_tag, которые присутствуют в наборе оперонов.

Результат — таблица соответствий: ген оперона (locus_tag) -> идентификатор
белка RefSeq (refseq_protein_id), нужная для последующего объединения
с UniProt.

Вход:  operon_genes_RS.parquet, selected_assemblies_with_gff.parquet
Выход: refseq_locus_to_protein.parquet
"""


import pandas as pd
import gzip
import requests
from io import BytesIO



operon = pd.read_parquet("operon_genes_RS.parquet")
operon["taxon_id"] = operon["taxon_id"].astype(int)

assemblies = pd.read_parquet("selected_assemblies_with_gff.parquet")

results = []

for _, row in assemblies.iterrows():

    taxid = row["taxid"]
    url = row["gff_url"]

    print(f"Processing taxid {taxid}")

    # Для каждого организма формируем множество локус-тегов из оперонов:
    # это позволяет фильтровать GFF на лету, не сохраняя ненужные записи.
    locus_set = set(
        operon[operon["taxon_id"] == taxid]["gene_id"]
    )

    if len(locus_set) == 0:
        continue

    try:
        response = requests.get(url, timeout=60)
        response.raise_for_status()
        with gzip.open(BytesIO(response.content), "rt") as f:

            for line in f:
                # Комментарии в GFF начинаются с #; пропускаем их.
                if line.startswith("#"):
                    continue

                parts = line.strip().split("\t")
```

```
                if len(parts) < 9:
                    continue

                # Нас интересуют только записи типа CDS (кодирующие участки),
                # поскольку для них определён identifier белка.
                if parts[2] != "CDS":
                    continue

                attrs = parts[8]

                if "locus_tag=" not in attrs:
                    continue

                # Атрибуты в GFF-формате: key1=value1;key2=value2;...
                # Разбираем в словарь для прямого доступа.
                attr_dict = dict(
                    field.split("=", 1)
                    for field in attrs.split(";")
                    if "=" in field
                )

                locus = attr_dict.get("locus_tag")
                protein = attr_dict.get("protein_id")

                if locus in locus_set:
                    results.append((taxid, locus, protein))

    except Exception as e:
        print(f"Failed {taxid}: {e}")

mapping_df = pd.DataFrame(
    results,
    columns=["taxid", "locus_tag", "refseq_protein_id"]
)

mapping_df.to_parquet("refseq_locus_to_protein.parquet", index=False)

print("Done. Rows:", len(mapping_df))
```

Приложение 5

```
"""
4UNIPROT_DOWNLOAD.py

Пакетная загрузка записей UniProtKB через REST API для всех организмов,
присутствующих в наборе оперонов. Из каждой записи извлекаются поля:
accession, organism_id, gene_names, xref_refseq, ec, sequence.

Реализованы механизмы отказоустойчивости: точки сохранения для возобновления
после прерывания и retry-логика при сетевых ошибках. Пакетирование по
15 taxid за запрос соблюдает ограничения UniProt API.

Вход:  operon_genes_RS.parquet
Выход: uniprot_with_refseq.tsv
       checkpoint_uniprot.txt — индекс последнего обработанного пакета
"""

import requests
import pandas as pd
from io import StringIO
import time
import os



operon_df = pd.read_parquet("operon_genes_RS.parquet")
taxids = operon_df["taxon_id"].astype(str).unique()

print("Total unique taxids:", len(taxids))

BASE_URL = "https://rest.uniprot.org/uniprotkb/stream"

# xref_refseq нужен для последующего объединения с RefSeq;
# ec и sequence — целевые поля для задачи предсказания EC-номеров.
FIELDS = "accession,organism_id,gene_names,xref_refseq,ec,sequence"

batch_size = 15
output_file = "uniprot_with_refseq.tsv"
checkpoint_file = "checkpoint_uniprot.txt"

# Восстановление позиции из точки сохранения при возобновлении.
start_index = 0

if os.path.exists(checkpoint_file):
    with open(checkpoint_file) as f:
        start_index = int(f.read().strip())
    print(f"Resuming from batch index {start_index}")
else:
    print("Starting from scratch")

file_exists = os.path.exists(output_file)

for i in range(start_index, len(taxids), batch_size):

    batch = taxids[i:i+batch_size]

    # Запрос UniProt в формате OR-выражения по organism_id; такой синтаксис
```

```
    # позволяет получить все белки нескольких организмов одним запросом.
    query = "(" + " OR ".join([f"organism_id:{t}" for t in batch]) + ")"

    params = {
        "format": "tsv",
        "fields": FIELDS,
        "query": query
    }

    print(f"Downloading batch {i}-{i+len(batch)}")

    try:
        response = requests.get(BASE_URL, params=params, timeout=300)
        response.raise_for_status()

        if response.text.strip():

            df = pd.read_csv(StringIO(response.text), sep="\t")

            if not df.empty:

                df.to_csv(
                    output_file,
                    sep="\t",
                    index=False,
                    mode="a",
                    header=not file_exists
                )

                file_exists = True

        # Точка сохранения обновляется только после успешной записи пакета.
        with open(checkpoint_file, "w") as f:
            f.write(str(i + batch_size))

        # Пауза между запросами для соблюдения ограничений UniProt API.
        time.sleep(1.5)

    except Exception as e:
        print(f"Error in batch {i}: {e}")
        print("Retrying in 15 seconds...")
        time.sleep(15)
        continue

print("Download finished.")
```

Приложение 6

```
"""
5UNIPROT_FILTER.py

Фильтрация загруженных записей UniProt: удаление записей без RefSeq,
разворачивание множественных RefSeq в длинный формат, отбор только
идентификаторов формата WP_* (RefSeq Non-redundant Protein), нормализация
поля EC number и удаление дубликатов.

Идентификаторы WP_* выбраны как стандартный формат RefSeq для прокариотических
белков; другие префиксы (NP_, YP_) относятся к специфическим сборкам и для
кросс-видового анализа менее применимы.

Вход:  uniprot_with_refseq.tsv
Выход: uniprot_with_refseq_clean.parquet
"""


import pandas as pd


print("Loading UniProt...")


df = pd.read_csv("uniprot_with_refseq.tsv", sep="\t")
print("Initial shape:", df.shape)


# Записи без RefSeq невозможно сопоставить с оперонными данными,
# поэтому исключаются на первом этапе.
df = df[df["RefSeq"].notna()]
df = df[df["RefSeq"] != ""]
print("After removing empty RefSeq:", df.shape)


# Поле xref_refseq в UniProt может содержать несколько идентификаторов
# через ";" или пробел; разворачиваем в длинный формат для последующего
# поэлементного объединения с RefSeq.
df["RefSeq"] = (
    df["RefSeq"]
    .astype(str)
    .str.replace(";", " ")
    .str.split()
)


df = df.explode("RefSeq")
df["RefSeq"] = df["RefSeq"].str.strip()
print("After explode:", df.shape)


# Фильтрация по префиксу WP_: оставляем только записи стандартного формата
# RefSeq Non-redundant Protein.
df = df[df["RefSeq"].str.startswith("WP_")]
print("After keeping WP_ only:", df.shape)


# Нормализация EC-номеров: преобразование строки разделённых значений
# в список для последующей обработки. Пустые значения сохраняются как
# пустые списки и фильтруются на следующем шаге.
df["EC number"] = (
    df["EC number"]
    .fillna("")
    .astype(str)
    .str.replace(";", " ")
    .str.split()
)
```

```
# Удаляем записи без аннотации EC-номеров: для задачи предсказания
# EC-номеров такие записи нерелевантны.
df = df[df["EC number"].str.len() > 0]
print("After EC cleaning:", df.shape)


# Дедупликация по паре (Entry, RefSeq) гарантирует уникальность ключа
# для последующего объединения с данными о локус-тегах.
df = df.drop_duplicates(
    subset=["Entry", "RefSeq"]
)
print("After deduplication:", df.shape)


df.to_parquet(
    "uniprot_with_refseq_clean.parquet",
    index=False
)
print("Saved cleaned UniProt file.")
```

Приложение 7

```
"""
6MERGE_FINAL.py

Финальное объединение трёх источников данных конвейера: оперонной разметки
(ODB), соответствий locus_tag -> protein_id (NCBI RefSeq GFF) и записей
UniProt с EC-номерами и аминокислотными последовательностями.

Объединение выполняется в два этапа:
  1. operon + locus->RefSeq по паре (taxon_id, gene_id);
  2. результат + UniProt по идентификатору белка RefSeq.

В обоих случаях используется inner join — для контроля потерь рассчитывается
покрытие на каждом этапе. После объединения удаляются дубликаты по паре
(Entry, operon_id) и отфильтровываются опероны, содержащие менее двух
белков, поскольку для них оперонный контекст неприменим.

Вход:  operon_genes_RS.parquet, refseq_locus_to_protein.parquet,
       uniprot_with_refseq_clean.parquet
Выход: merged_operon_uniprot_refseq_filtered.parquet
"""


import pandas as pd
import numpy as np
from collections import defaultdict


print("Loading files...")


operon = pd.read_parquet("operon_genes_RS.parquet")
operon["taxon_id"] = operon["taxon_id"].astype(str)


refseq_map = pd.read_parquet("refseq_locus_to_protein.parquet")
refseq_map["taxid"] = refseq_map["taxid"].astype(str)


# Приведение имён колонок к общему виду для прямого объединения.
refseq_map = refseq_map.rename(columns={
    "taxid": "taxon_id",
    "locus_tag": "gene_id"
})


uniprot = pd.read_parquet("uniprot_with_refseq_clean.parquet")


print("Operon:", operon.shape)
print("RefSeq map:", refseq_map.shape)
print("UniProt:", uniprot.shape)


# Этап 1: оперонные данные + соответствия locus_tag -> protein_id.
# Inner join даёт потерю генов, для которых не удалось установить
# идентификатор белка RefSeq (несоответствие между ODB и актуальными
# GFF-аннотациями, отсутствие записей кодирующих последователь
кодирующих последовательстей для некоторых локус-тегов).
print("\nMerging operon with RefSeq mapping...")
operon_refseq = operon.merge(
    refseq_map,
    on=["taxon_id", "gene_id"],
    how="inner"
)
print("After operon RefSeq:", operon_refseq.shape)
```

```
# Этап 2: добавление данных UniProt по идентификатору белка RefSeq.
# Здесь же подтягиваются аминокислотные последовательности и EC-номера —
# целевые поля для задачи предсказания.
print("\nMerging with UniProt...")
merged = operon_refseq.merge(
    uniprot,
    left_on="refseq_protein_id",
    right_on="RefSeq",
    how="inner"
)
print("After RefSeq UniProt:", merged.shape)


merged = merged[
    ["Entry", "taxon_id", "operon_id",
     "refseq_protein_id", "EC number", "Sequence"]
]


# Один белок может встретиться в опероне несколько раз через разные
# RefSeq-идентификаторы; дедупликация по паре (Entry, operon_id)
# обеспечивает уникальность вхождения белка в каждый оперон.
merged = merged.drop_duplicates(
    subset=["Entry", "operon_id"]
)
print("After dedup:", merged.shape)


# Фильтрация оперонов по количеству белков: для оперонного контекста
# необходим хотя бы один сосед, поэтому опероны из одного белка
# исключаются как нерелевантные.
print("\nFiltering single-gene operons...")
operon_counts = defaultdict(set)


for op, entry in zip(merged["operon_id"], merged["Entry"]):
    operon_counts[op].add(entry)


valid_operons = {
    op for op, entries in operon_counts.items()
    if len(entries) >= 2
}


merged_filtered = merged[
    merged["operon_id"].isin(valid_operons)
]


print("Before filter:", merged.shape)
print("After filter:", merged_filtered.shape)


merged_filtered.to_parquet(
    "merged_operon_uniprot_refseq_filtered.parquet",
    index=False
)
print("\nSaved merged_operon_uniprot_refseq_filtered.parquet")


# Сводная статистика по покрытию и характеристикам итогового набора.
print(f"\nПокрытие operon RefSeq: {len(operon_refseq)/len(operon)*100:.1f}%")
print(f"Покрытие RefSeq UniProt: {len(merged)/len(operon_refseq)*100:.1f}%")
print(f"Общее покрытие от исходных оперонов: "
      f"{len(merged_filtered)/len(operon)*100:.1f}%")


print(f"\nУникальных белков (Entry): {merged_filtered['Entry'].nunique()}")
print(f"Уникальных оперонов: {merged_filtered['operon_id'].nunique()}")
print(f"Уникальных организмов: {merged_filtered['taxon_id'].nunique()}")
```

```
print(f"Уникальных EC-номеров: "
      f"{merged_filtered['EC number'].explode().nunique()}")


op_sizes = merged_filtered.groupby("operon_id")["Entry"].nunique()
print(f"\nРазмеры оперонов (белков на оперон):")
print(f"  min: {op_sizes.min()}, max: {op_sizes.max()}, "
      f"median: {op_sizes.median():.0f}, mean: {op_sizes.mean():.1f}")


seq_lens = merged_filtered["Sequence"].str.len()
print(f"\nДлины последовательностей (а.к.):")
print(f"  min: {seq_lens.min()}, max: {seq_lens.max()}, "
      f"median: {seq_lens.median():.0f}, mean: {seq_lens.mean():.0f}")


ec_counts = merged_filtered["EC number"].apply(
    lambda x: len(x) if isinstance(x, list) else 1
)
print(f"\nБелков с 1 EC: {(ec_counts == 1).sum()}")
print(f"Белков с >=2 EC: {(ec_counts >= 2).sum()} "
      f"({(ec_counts >= 2).mean()*100:.1f}%)")
```

Приложение 8

Скрипт реализует оценку моделей предсказания EC-номеров на построенном оценочном наборе. Он выполняет построение оценочной выборки с дедупликацией граничных генов, стратификацию по уровню гомологии с обучающей выборкой средствами MMseqs2, сквозное сканирование по порогу уверенности с автоматическим выбором оптимального порога для каждой модели, а также анализ эффективности оперонного фильтра по стратам гомологии и уровням иерархии EC-номеров.

```
"""
7DIPLOMA_EVALUATION.py

Единый скрипт оценки моделей предсказания EC-номеров. Реализует три блока:
  2.1  Базовая оценка ProteInfer и CLEAN: сквозное сканирование по порогу
       уверенности и автоматический выбор оптимального
       порога для каждой модели по критерию максимума F1 macro после
       применения оперонного фильтра.
  2.2  Анализ эффективности оперонного фильтра: прирост F1 macro
       по стратам гомологии (HIGH/MEDIUM/LOW/NONE) и по уровням иерархии
       EC-номеров (L1..L4).
  2.4  Анализ независимости уверенности модели и оперонного контекста:
       какую долю ошибок при высокой уверенности предсказаний модели ловит
       оперонный фильтр в зависимости от страты гомологии.

Критерий выбора порога идентичен для обеих моделей: максимум F1 macro после
применения оперонного фильтра. Тот же порог затем используется
во всех последующих анализах.

Вход:
  merged_operon_uniprot_refseq_filtered.parquet  - оценочный набор оперонов
  pathid_ec.tsv                                  - соответствия EC -> KEGG path
  research_proteins_..._train.tfrecord           - train ProteInfer
  split100.csv                                   - train CLEAN
  proteinfer_predictions2.tsv                    - предсказания ProteInfer
  query_maxsep.csv                               - предсказания CLEAN

Выход (в RESULTS_DIR):
  sec21_threshold_sweep.csv          - метрики при всех порогах для обеих моделей
  sec21_optimal_threshold.csv        - выбранные оптимальные пороги
  sec21_baselines.csv  / .png        - сравнение моделей по стратам
  sec21_pareto.png                   - Pareto-фронт coverage / F1
  sec22_binary_delta_f1.png          - прирост F1 после фильтра по стратам
  sec22_f1_by_ec_level.png           - F1 по уровням EC-иерархии
  sec22_ec_class_heatmap.png         - F1 по классам EC (первая цифра)
  sec24_high_conf_errors.png         - ошибки модели и recall фильтра по стратам
"""

import os
import struct
import subprocess
import warnings
from collections import defaultdict
```

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd


warnings.filterwarnings("ignore")



# КОНФИГУРАЦИЯ


BASE_DIR = "/mnt/storage/home/sykarmanov/proteinfer"
OUTPUT_DIR = os.path.join(BASE_DIR, "evaluation2")
RESULTS_DIR = os.path.join(OUTPUT_DIR, "results1")


PARQUET_PATH = os.path.join(BASE_DIR,
    "merged_operon_uniprot_refseq_filtered.parquet")
PATHID_EC_PATH = os.path.join(BASE_DIR, "preprocessing", "pathid_ec.tsv")
PI_TFRECORD_PATH = os.path.join(BASE_DIR,
    "research_proteins_proteinfer_datasets_swissprot_clustered_train.tfrecord")
CLEAN_TRAIN_PATH = os.path.join(BASE_DIR, "split100.csv")
PI_PRED_PATH = os.path.join(BASE_DIR, "proteinfer_predictions2.tsv")
CLEAN_PRED_PATH = os.path.join(BASE_DIR, "query_maxsep.csv")


MMSEQS_BIN = "/mnt/storage/home/sykarmanov/mmseqs/bin/mmseqs"
MMSEQS_WORK_DIR = os.path.join(OUTPUT_DIR, "mmseqs_work")
EVAL_FASTA_PATH = os.path.join(OUTPUT_DIR, "eval_sequences.fasta")
TRAIN_FASTA_PATH = os.path.join(OUTPUT_DIR, "train_combined.fasta")


# Границы страт по максимальной идентичности с обучающей выборкой;
# страты соответствуют стандартному разделению на гомологичные группы.
IDENTITY_THRESHOLDS = {"HIGH": 0.70, "MEDIUM": 0.40}
BLACKLIST_TOP_N = 3
STRATA_ORDER = ["HIGH", "MEDIUM", "LOW", "NONE"]


# Сигнальное значение для белков без опероннного контекста: их предсказания
# проходят бинарный фильтр без проверки.
NO_CONTEXT_SCORE = float("inf")
BINARY_TAU = 0.5
CONF_THRESHOLDS = [0.0, 0.3, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95]
CLEAN_PARSE_MIN = 1e-9


# CLEAN: порог выбирается по тому же критерию, что и для ProteInfer -
# максимум F1 macro после опероннного фильтра. Константа оставлена
# как резервный вариант для ручной установки: число – фиксированный порог, None -
автоматический выбор.
CLEAN_FIXED_THR = None



# ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ


def normalize_ec(s):
    return s.strip().replace("EC:", "").replace("ec:", "").strip()



def is_full_ec(ec):
    p = ec.split(".")
    return len(p) == 4 and all(x.isdigit() for x in p)
```

```
def ec_at_level(ec, level):
    return ".".join(ec.split(".")[:level])



# ЗАГРУЗКА TRAIN


def parse_tfrecord_sequences(path):
    """Извлечение последовательностей из TFRecord ProteInfer.

    Сначала пытается использовать TensorFlow API. При его отсутствии
    выполняется ручной разбор protobuf-формата TFRecord для извлечения
    поля 'sequence' без зависимости от тяжёлой библиотеки.
    """
    print(f"\n  Загрузка train ProteInfer: {path}")
    sequences = set()
    try:
        import tensorflow as tf
        raw = tf.data.TFRecordDataset(path)
        spec = {"sequence": tf.io.FixedLenFeature([], tf.string)}
        cnt = 0
        for rec in raw:
            seq = tf.io.parse_single_example(rec, spec)["sequence"]
            seq = seq.numpy().decode("utf-8")
            sequences.add(seq); cnt += 1
            if cnt % 50000 == 0: print(f"      {cnt} записей...")
        print(f"    {len(sequences)} уникальных из {cnt}")
        return sequences
    except ImportError:
        print("    Ручной парсинг protobuf...")


    # Ручной разбор protobuf-формата TFRecord. Функции с короткими именами
    # реализуют примитивы декодирования: чтение записи (_rd), varint (_dv),
    # упакованного байт-списка (_pbl), отдельного поля (_pf), записи
    # MapEntry (_pme), словарь признаков (_pfs) и Example (_pe).
    def _rd(f):
        b = f.read(8)
        if len(b) < 8: return None
        ln = struct.unpack("<Q", b)[0]; f.read(4)
        d = f.read(ln); f.read(4)
        return d


    def _dv(d, p):
        r = s = 0
        while p < len(d):
            b = d[p]; p += 1; r |= (b & 0x7F) << s
            if not (b & 0x80): break
            s += 7
        return r, p


    def _pbl(d):
        v, p = [], 0
        while p < len(d):
            t, p = _dv(d, p)
            if (t & 7) == 2:
                l, p = _dv(d, p); v.append(d[p:p+l]); p += l
            else:
                break
        return v


    def _pf(d):
        p = 0
        while p < len(d):
```

```
        t, p = _dv(d, p)
        if (t & 7) == 2:
            l, p = _dv(d, p); pl = d[p:p+l]; p += l
            if (t >> 3) == 1: return _pbl(pl)
        elif (t & 7) == 0:
            _, p = _dv(d, p)
        else:
            break

def _pme(d):
    k = v = None; p = 0
    while p < len(d):
        t, p = _dv(d, p)
        if (t & 7) == 2:
            l, p = _dv(d, p); pl = d[p:p+l]; p += l
            if (t >> 3) == 1: k = pl.decode("utf-8")
            elif (t >> 3) == 2: v = _pf(pl)
        elif (t & 7) == 0:
            _, p = _dv(d, p)
        else:
            break
    return k, v

def _pfs(d):
    fs, p = {}, 0
    while p < len(d):
        t, p = _dv(d, p)
        if (t & 7) == 2 and (t >> 3) == 1:
            l, p = _dv(d, p); k2, v2 = _pme(d[p:p+l]); p += l
            if k2: fs[k2] = v2
        elif (t & 7) == 2:
            l, p = _dv(d, p); p += l
        elif (t & 7) == 0:
            _, p = _dv(d, p)
        else:
            break
    return fs

def _pe(d):
    fs, p = {}, 0
    while p < len(d):
        t, p = _dv(d, p)
        if (t & 7) == 2:
            l, p = _dv(d, p)
            if (t >> 3) == 1: fs = _pfs(d[p:p+l])
            p += l
        elif (t & 7) == 0:
            _, p = _dv(d, p)
        else:
            break
    return fs

cnt = 0
with open(path, "rb") as f:
    while True:
        rec = _rd(f)
        if rec is None: break
        try:
            fs = _pe(rec)
            if "sequence" in fs and fs["sequence"]:
                sb = fs["sequence"]
                if isinstance(sb, list) and sb:
                    sq = sb[0].decode("utf-8")
                elif isinstance(sb, bytes):
                    sq = sb.decode("utf-8")
                else:
```

```
                        sq = None
                    if sq: sequences.add(sq)
            except: pass
            cnt += 1
            if cnt % 50000 == 0: print(f"      {cnt} записей...")
    print(f"    {len(sequences)} уникальных из {cnt}")
    return sequences



def load_clean_train(path):
    print(f"\n  Загрузка train CLEAN: {path}")
    df = pd.read_csv(path, sep="\t")
    col = next((c for c in ["Sequence", "sequence", "seq"]
                if c in df.columns), None)
    if not col: return set()
    seqs = set(df[col].dropna().astype(str).str.strip())
    print(f"    {len(seqs)} последовательностей")
    return seqs



def load_all_train():
    print(f"\n{'='*70}\nЗагрузка train")
    pi = (parse_tfrecord_sequences(PI_TFRECORD_PATH)
          if os.path.isfile(PI_TFRECORD_PATH) else set())
    cl = (load_clean_train(CLEAN_TRAIN_PATH)
          if os.path.isfile(CLEAN_TRAIN_PATH) else set())
    # Объединение train ProteInfer и train CLEAN: белок считается тренировочным
    # для последующего исключения из eval, если он присутствует хотя бы в одном
    # из двух наборов (правило строгого исключения по последовательности).
    t = pi | cl
    print(f"\n  PI: {len(pi)}, CLEAN: {len(cl)}, "
          f"пересечение: {len(pi&cl)}, всего: {len(t)}")
    return t



# ПОСТРОЕНИЕ ОЦЕНОЧНОГО ДАТАСЕТА

def _boundary_gene_dedup(df):
    """Дедупликация граничных генов: каждый белок (Entry) в пределах
    организма (taxon_id) привязывается ровно к одному оперону.

    Выбирается оперон наибольшего размера; при равенстве размеров -
    стабильно по operon_id. Без этого этапа граничные гены, входящие
    одновременно в несколько перекрывающихся оперонов, дают завышение
    числа уникальных оперонов на величину их кратности.
    """
    op_key = ["operon_id", "taxon_id"]
    op_size = (df.drop_duplicates(subset=op_key + ["Entry"])
                 .groupby(op_key, observed=True)["Entry"]
                 .size()
                 .rename("op_size")
                 .reset_index())
    pairs = (df.drop_duplicates(subset=["Entry"] + op_key)
               .merge(op_size, on=op_key, how="left", validate="m:1"))
    pairs = pairs.sort_values(["op_size", "operon_id"],
                              ascending=[False, True])
    kept = pairs.drop_duplicates(subset=["Entry", "taxon_id"],
                                 keep="first")
    chosen = kept[["Entry", "operon_id", "taxon_id"]]
    return df.merge(chosen, on=["Entry", "operon_id", "taxon_id"],
                    how="inner")
```

```
def _drop_operon_singletons(df, op_key=("operon_id", "taxon_id"),
                            value_col="Entry"):
    """Удаление оперонов, содержащих менее двух уникальных значений
    value_col. Возвращает отфильтрованный df и число удалённых одногенных оперонов.
    """
    op_key = list(op_key)
    sz = (df.drop_duplicates(subset=[value_col] + op_key)
            .groupby(op_key, observed=True)[value_col]
            .size())
    valid = sz[sz >= 2].index
    n_removed = int((sz == 1).sum())
    df = df.set_index(op_key)
    df = df[df.index.isin(valid)].reset_index()
    return df, n_removed



def _n_op(df):
    """Число уникальных пар (operon_id, taxon_id) - идентификатор оперона."""
    return df.drop_duplicates(subset=["operon_id", "taxon_id"]).shape[0]



def build_eval_dataset(train_sequences):
    """Каскадное построение оценочного набора по этапам, описанным в 2.5
    методической части. На каждом этапе печатается число белков и оперонов
    для последующей сверки с дипломом.
    """
    print(f"\n{'#'*70}\n# ПОСТРОЕНИЕ EVAL-ДАТАСЕТА "
          f"(дедуп граничных генов на входе)\n{'#'*70}")
    df = pd.read_parquet(PARQUET_PATH)
    print(f"\n  Загружено: {len(df)} строк, "
          f"{df['Entry'].nunique()} Entry, "
          f"{_n_op(df)} пар operon_id+taxon_id")


    print(f"\n  --- Дедуп граничных генов (вход) ---")
    df = _boundary_gene_dedup(df)
    df, n_art = _drop_operon_singletons(df, value_col="Entry")
    print(f"  Синглтонов-артефактов убрано: {n_art}")
    print(f"  После дедупа: {df['Entry'].nunique()} белков, "
          f"{_n_op(df)} оперонов")


    # EC-номера приводятся к кортежу для последующего разворачивания (explode).
    # Поле может быть строкой (одно EC), списком, numpy-массивом или NaN -
    # все варианты нормализуются к виду tuple.
    df["EC number"] = df["EC number"].apply(
        lambda x: tuple(x) if isinstance(x, (np.ndarray, list, tuple))
        else (x,))


    # Строгое исключение: удаляются ВСЕ белки, последовательность которых
    # встретилась хотя бы в одной обучающей выборке. Это гарантирует
    # отсутствие утечки и позволяет корректно интерпретировать результаты.
    if train_sequences:
        m = df["Sequence"].isin(train_sequences)
        print(f"\n  Train: удалено {m.sum()} строк")
        df = df[~m].copy()
        print(f"  После вычета train: {df['Entry'].nunique()} белков, "
              f"{_n_op(df)} оперонов")


    df = df.drop_duplicates(
        subset=["Sequence", "operon_id", "taxon_id"]).copy()
    print(f"  После drop_duplicates: {df['Entry'].nunique()} белков, "
          f"{_n_op(df)} оперонов")
```

```
df, n_sg = _drop_operon_singletons(df, value_col="Sequence")
print(f"  Одногенных оперонов после вычета train: {n_sg}")
print(f"  После удаления одногенный оперонов: {df['Entry'].nunique()} белков, "
      f"{_n_op(df)} оперонов")


df = df.explode("EC number").copy()
df["EC number"] = df["EC number"].astype(str)
df = df[(df["EC number"] != "nan")
        & (df["EC number"].str.strip() != "")].copy()


# Обогащение оценочного набора аннотациями метаболических путей KEGG.
print(f"\n  --- Обогащение метаболическими путями ---")
pathid = pd.read_csv(PATHID_EC_PATH, sep="\t")
print(f"  Файл pathid_ec.tsv:")
print(f"    Записей (EC -> Path): {len(pathid)}")
print(f"    Уникальных EC-номеров: {pathid['EC_Numbers'].nunique()}")
print(f"    Уникальных путей KEGG: {pathid['Path'].nunique()}")


ec_in_dataset = set(df["EC number"].unique())
ec_in_pathid = set(pathid["EC_Numbers"].unique())
ec_covered = ec_in_dataset & ec_in_pathid
ec_not_covered = ec_in_dataset - ec_in_pathid
print(f"  Покрытие EC-номеров датасета:")
print(f"    EC в датасете: {len(ec_in_dataset)}")
print(f"    EC с путями: {len(ec_covered)} "
      f"({100*len(ec_covered)/len(ec_in_dataset):.1f}%)")
print(f"    EC без путей: {len(ec_not_covered)} "
      f"({100*len(ec_not_covered)/len(ec_in_dataset):.1f}%)")


df = df.merge(pathid, left_on="EC number",
              right_on="EC_Numbers", how="left")
if "EC_Numbers" in df.columns:
    df = df.drop(columns=["EC_Numbers"])


n_with_path = df["Path"].notna().sum()
n_without_path = df["Path"].isna().sum()
print(f"  Покрытие записей:")
print(f"    Записей с путями: {n_with_path} "
      f"({100*n_with_path/len(df):.1f}%)")
print(f"    Записей без путей: {n_without_path} "
      f"({100*n_without_path/len(df):.1f}%)")


# Удаление неинформативных оперонов: оперонов, в которых после фильтрации
# по наличию аннотации Path осталось менее двух белков. Такие опероны
# не предоставляют контекста соседних генов для фильтра.
print(f"\n  --- Удаление неинформативных оперонов ---")
df_wp = df[df["Path"].notna()].copy()
ppo = (df_wp.drop_duplicates(subset=["Entry", "operon_id", "taxon_id"])
            .groupby(["operon_id", "taxon_id"], observed=True)["Entry"]
            .nunique())
valid = set(ppo[ppo >= 2].index)
df_idx = df.set_index(["operon_id", "taxon_id"])
all_ops = set(df_idx.index.unique())
eff_sg = all_ops - valid


n_before = _n_op(df)
n_proteins_before = df["Entry"].nunique()
df = df_idx[df_idx.index.isin(list(valid))].reset_index()
n_after = _n_op(df)
n_proteins_after = df["Entry"].nunique()


print(f"  До фильтрации:")
```

```
    print(f"    Оперонов: {n_before}")
    print(f"    Белков: {n_proteins_before}")
    print(f" Неинформативных оперонов "
          f"(< 2 белков с путями): {len(eff_sg)}")
    print(f"  После фильтрации:")
    print(f"    Оперонов: {n_after} ({100*n_after/n_before:.1f}%)")
    print(f"    Белков: {n_proteins_after} "
          f"({100*n_proteins_after/n_proteins_before:.1f}%)")


    # Финальная дедупликация по тройке (Sequence, operon_id, Path).
    # Заполнение NaN маркером __NO_PATH__ нужно, чтобы записи без аннотации
    # пути не схлопывались в дубликаты по двум первым полям.
    df["_pk"] = df["Path"].fillna("__NO_PATH__")
    df = df.drop_duplicates(
        subset=["Sequence", "operon_id", "_pk"]).drop(columns=["_pk"]).copy()
    print(f"  Итог: {len(df)} строк, {df['Sequence'].nunique()} послед., "
          f"{_n_op(df)} оперонов")


    # FASTA для MMseqs2: уникальные последовательности с идентификатором Entry.
    uq = df.drop_duplicates(subset=["Sequence"])[["Entry", "Sequence"]]
    with open(EVAL_FASTA_PATH, "w") as f:
        for _, r in uq.iterrows():
            f.write(f">{r['Entry']}\n{r['Sequence']}\n")
    print(f"  FASTA: {len(uq)} послед.")
    return df



def save_train_fasta(train):
    with open(TRAIN_FASTA_PATH, "w") as f:
        for i, s in enumerate(train):
            f.write(f">train_{i}\n{s}\n")
    print(f"  Train FASTA: {len(train)} послед.")



# СТРАТИФИКАЦИЯ ПО ГОМОЛОГИИ (MMseqs2)


def run_mmseqs2(eval_df):
    """Сравнение всех eval-последовательностей со всеми train для определения
    максимальной идентичности и разделения eval на страты HIGH/MEDIUM/LOW/NONE.


    Использует кэш в MMSEQS_WORK_DIR: если result.tsv существует и не пуст,
    повторный поиск не запускается. При отсутствии MMseqs2 в системе все
    последовательности отнесены к страте ALL.
    """
    print(f"\n{'#'*70}\n# СТРАТИФИКАЦИЯ (MMSeqs2)\n{'#'*70}")
    os.makedirs(MMSEQS_WORK_DIR, exist_ok=True)
    result_tsv = os.path.join(MMSEQS_WORK_DIR, "result.tsv")


    if os.path.isfile(result_tsv) and os.path.getsize(result_tsv) > 0:
        print(f"  [CACHE] result.tsv "
              f"({os.path.getsize(result_tsv)/1e6:.1f} MB)")
    else:
        if not os.path.isfile(MMSEQS_BIN):
            return _fallback_strata(eval_df)
        import shutil
        # Чистим рабочую директорию перед новым поиском.
        for it in os.listdir(MMSEQS_WORK_DIR):
            p = os.path.join(MMSEQS_WORK_DIR, it)
            shutil.rmtree(p) if os.path.isdir(p) else os.remove(p)


        tdb = os.path.join(MMSEQS_WORK_DIR, "train_db")
        edb = os.path.join(MMSEQS_WORK_DIR, "eval_db")
```

```
    rdb = os.path.join(MMSEQS_WORK_DIR, "result")
    tmp = os.path.join(MMSEQS_WORK_DIR, "tmp")
    os.makedirs(tmp, exist_ok=True)


    def run_cmd(cmd, desc):
        print(f"  [{desc}]")
        r = subprocess.run(cmd, capture_output=True, text=True)
        if r.returncode:
            print(f"  [ERR] {r.stderr[:300]}")
            return False
        return True


    # Параметры поиска: высокая чувствительность s=7.5 для нахождения
    # отдалённых гомологов; e-value 10 (мягкий порог) - реальный
    # критерий принадлежности к страте задаётся позже через идентичность.
    commands = [
        ([MMSEQS_BIN, "createdb", TRAIN_FASTA_PATH, tdb],
         "createdb train"),
        ([MMSEQS_BIN, "createdb", EVAL_FASTA_PATH, edb],
         "createdb eval"),
        ([MMSEQS_BIN, "search", edb, tdb, rdb, tmp,
          "--min-seq-id", "0.0", "-s", "7.5",
          "--max-seqs", "300", "-e", "10", "--threads", "8"],
         "search"),
        ([MMSEQS_BIN, "convertalis", edb, tdb, rdb, result_tsv,
          "--format-output", "query,target,fident,alnlen,evalue"],
         "convertalis"),
    ]
    for cmd, desc in commands:
        if not run_cmd(cmd, desc):
            return _fallback_strata(eval_df)

if not os.path.isfile(result_tsv) or os.path.getsize(result_tsv) == 0:
    return _fallback_strata(eval_df)


hits = pd.read_csv(result_tsv, sep="\t", header=None,
                   names=["query", "target", "fident", "alnlen", "evalue"])
print(f"  Хитов: {len(hits)}, query: {hits['query'].nunique()}")


# Для каждого eval-белка - максимальная идентичность с любым train-белком.
mi = hits.groupby("query")["fident"].max().reset_index()
mi.columns = ["Entry", "max_identity"]
bh = hits.loc[hits.groupby("query")["fident"].idxmax()][
    ["query", "target", "fident"]]
bh.columns = ["Entry", "nearest_train_entry", "best_fident"]
info = mi.merge(bh[["Entry", "nearest_train_entry"]],
                on="Entry", how="left")


def assign_group(x):
    # NONE - ни одного хита (даже мягкого) с train.
    # HIGH/MEDIUM/LOW - по границам, заданным в IDENTITY_THRESHOLDS.
    if pd.isna(x): return "NONE"
    if x > IDENTITY_THRESHOLDS["HIGH"]: return "HIGH"
    if x >= IDENTITY_THRESHOLDS["MEDIUM"]: return "MEDIUM"
    return "LOW"


info["identity_group"] = info["max_identity"].apply(assign_group)
eval_df = eval_df.merge(info, on="Entry", how="left")


# Белки без хитов не попадают в info после merge; заполняем по умолчанию.
for col, val in [("max_identity", 0.0),
                 ("identity_group", "NONE"),
                 ("nearest_train_entry", "")]:
```

```
        eval_df[col] = eval_df[col].fillna(val)

    gc = eval_df.drop_duplicates("Entry").groupby("identity_group").size()
    tot = eval_df["Entry"].nunique()
    for g in STRATA_ORDER:
        print(f"  {g:>6}: {gc.get(g,0):>6} ({100*gc.get(g,0)/tot:.1f}%)")
    return eval_df



def _fallback_strata(df):
    print("  [FALLBACK] -> ALL")
    df["max_identity"] = np.nan
    df["identity_group"] = "ALL"
    df["nearest_train_entry"] = ""
    return df



# РАЗБОР ПРЕДСКАЗАНИЙ


def parse_pi_predictions(path):
    """Разбор предсказаний ProteInfer. Из каждой строки извлекаются
    идентификатор белка, предсказанный EC-номер и уверенность модели.
    Внутри одного белка предсказания сортируются по убыванию уверенности.
    """
    print(f"\n  Разбор ProteInfer: {path}")
    df = pd.read_csv(path, sep="\t")
    preds = defaultdict(list)
    for _, row in df.iterrows():
        ec = normalize_ec(str(row["predicted_label"]))
        if is_full_ec(ec):
            preds[str(row["sequence_name"]).strip()].append(
                (ec, float(row["confidence"])))
    for e in preds:
        preds[e].sort(key=lambda x: -x[1])
    print(f"    Белков: {len(preds)}")
    return dict(preds)



def parse_clean_predictions(path, min_conf=CLEAN_PARSE_MIN):
    """Разбор предсказаний CLEAN. Формат - CSV: на каждой строке белок
    и список предсказаний вида EC/score через запятую. Предсказания с
    уверенностью ниже min_conf отбрасываются на этапе разбора.
    """
    print(f"\n  Разбор CLEAN: {path} (min_conf={min_conf})")
    preds = defaultdict(list)
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line: continue
            parts = line.split(",")
            entry = parts[0].strip()
            for part in parts[1:]:
                part = part.strip()
                if "/" not in part: continue
                ec_part, score_part = part.rsplit("/", 1)
                try:
                    score = float(score_part)
                except ValueError:
                    continue
                if score < min_conf: continue
                ec = normalize_ec(ec_part)
                if is_full_ec(ec):
                    preds[entry].append((ec, score))
```

```
    for e in preds:
        preds[e].sort(key=lambda x: -x[1])
    print(f"    Белков: {len(preds)}")
    return dict(preds)



# КЭШ ПРЕДСКАЗАНИЙ

class PredictionCache:
    """Кэш для отфильтрованных по порогу предсказаний.

    Один и тот же порог фигурирует во многих секциях скрипта; чтобы
    не пересчитывать фильтрацию каждый раз, результаты для каждого
    значения порога кешируются. Аналогично для версий с применённым
    фильтром (pi_bin, cl_bin).
    """

    def __init__(self, pi_preds, clean_preds, all_entries, scorer):
        self.pi_raw = pi_preds
        self.cl_raw = clean_preds
        self.all_e = all_entries
        self.scorer = scorer
        self._pi_cache = {}
        self._cl_cache = {}
        self._pi_bin_cache = {}
        self._cl_bin_cache = {}

    def _filter(self, preds, thr):
        # Оставляем белков, у которых наиболее уверенное предсказание
        # имеет confidence >= thr и которые входят в оценочный набор.
        return {e: c for e, c in preds.items()
                if c and c[0][1] >= thr and e in self.all_e}

    def pi(self, thr):
        if thr not in self._pi_cache:
            self._pi_cache[thr] = self._filter(self.pi_raw, thr)
        return self._pi_cache[thr]

    def cl(self, thr):
        if thr not in self._cl_cache:
            self._cl_cache[thr] = self._filter(self.cl_raw, thr)
        return self._cl_cache[thr]

    def pi_bin(self, thr):
        if thr not in self._pi_bin_cache:
            self._pi_bin_cache[thr] = self.scorer.apply_binary_reject(
                self.pi(thr))
        return self._pi_bin_cache[thr]

    def cl_bin(self, thr):
        if thr not in self._cl_bin_cache:
            self._cl_bin_cache[thr] = self.scorer.apply_binary_reject(
                self.cl(thr))
        return self._cl_bin_cache[thr]



# ОПЕРОННЫЙ ФИЛЬТР

class OpeRONScorer:
    """Оперонный фильтр с адаптивным поведением: белки без
```

```
оперонного контекста проходят без проверки.


Для каждого белка фильтр проверяет два условия (раздел 2.9 методов):
  1) предсказанный EC-номер участвует хотя бы в одном метаболическом
     пути, общем с EC-номерами соседних белков по оперону;
  2) или совпадает с EC-номером хотя бы одного из соседей.
При выполнении любого из условий предсказание принимается.
"""


def __init__(self, eval_df, blacklist_top_n=BLACKLIST_TOP_N):
    print(f"\n{'='*70}\nOpeRONScorer "
          f"(adaptive reject, blacklist_top_n={blacklist_top_n})")
    self.eval_df = eval_df
    self.blacklist_top_n = blacklist_top_n
    self.ec_to_paths = self._build_ec_paths()
    self.operon_entry_paths = defaultdict(lambda: defaultdict(set))
    self.operon_entry_ecs = defaultdict(lambda: defaultdict(set))
    self.entry_to_operon = {}
    self._build()
    self._path_operon_count = defaultdict(int)
    self.blacklist = set()
    self._build_blacklist()
    self._print_stats()


def _build_ec_paths(self):
    # Соответствие EC-номера множеству KEGG-путей, в которых он участвует.
    r = defaultdict(set)
    for _, row in self.eval_df.iterrows():
        ec = normalize_ec(str(row["EC number"]))
        p = row.get("Path")
        if pd.notna(p) and str(p).strip():
            r[ec].add(str(p).strip())
    return dict(r)


def _build(self):
    # Индексы для быстрого поиска соседей и их аннотаций по идентификатору
    # оперона: пути и EC-номера каждого белка оперона.
    for _, row in self.eval_df.iterrows():
        oid, entry = row["operon_id"], row["Entry"]
        self.entry_to_operon[entry] = oid
        p = row.get("Path")
        if pd.notna(p) and str(p).strip():
            self.operon_entry_paths[oid][entry].add(str(p).strip())
        ec = normalize_ec(str(row["EC number"]))
        if is_full_ec(ec):
            self.operon_entry_ecs[oid][entry].add(ec)
    print(f"  Оперонов: {len(self.operon_entry_paths)}, "
          f"белков: {len(self.entry_to_operon)}")


def _build_blacklist(self):
    # Глобальные KEGG-карты (наиболее частые пути) уже исключены на этапе
    # 0PATHID_PREPROCESS.py. Здесь дополнительно отслеживаются топ-N
    # самых распространённых путей в текущем eval-наборе.
    for oid, ep in self.operon_entry_paths.items():
        ps = set()
        for e, paths in ep.items():
            ps.update(paths)
        for p in ps:
            self._path_operon_count[p] += 1
    sp = sorted(self._path_operon_count.items(), key=lambda x: -x[1])
    self.blacklist = set(p for p, _ in sp[:self.blacklist_top_n])
    print(f"  Чёрный список ({self.blacklist_top_n}):")
    for p in sorted(self.blacklist,
                    key=lambda x: -self._path_operon_count.get(x, 0)):
        print(f"    {p:>15}: {self._path_operon_count[p]} оперонов")
```

```
    def _print_stats(self):
        n_ctx = n_no = 0
        for entry, oid in self.entry_to_operon.items():
            if self._neighbor_tf(entry, oid):
                n_ctx += 1
            else:
                n_no += 1
        tot = n_ctx + n_no
        print(f"  С контекстом: {n_ctx} ({100*n_ctx/tot:.1f}%), "
              f"без: {n_no} ({100*n_no/tot:.1f}%)")
        print(f"  OpeRONScorer готов.\n")


    def _neighbor_tf(self, entry, oid):
        # Доли соседей оперона, аннотированных каждым из путей. Возвращает
        # словарь {path: доля соседей в этом пути} или пустой словарь,
        # если соседей с аннотациями в опероне нет.
        ep = self.operon_entry_paths.get(oid, {})
        nb = {e: ps for e, ps in ep.items() if e != entry}
        n = len(nb)
        if n == 0:
            return {}
        pc = defaultdict(int)
        for e, ps in nb.items():
            for p in ps:
                pc[p] += 1
        return {p: c/n for p, c in pc.items()}


    def _neighbor_ecs(self, entry, oid):
        # Объединение EC-номеров всех соседей оперона.
        r = set()
        for e, ecs in self.operon_entry_ecs.get(oid, {}).items():
            if e != entry:
                r.update(ecs)
        return r


    def score_binary(self, ec, entry):
        # Возвращает 1.0 если предсказание согласуется с оперонным контекстом
        # (общий путь или совпадение EC с соседом), 0.0 - если нет. Для белков
        # без контекста возвращает NO_CONTEXT_SCORE - обходящая фильтрацию.
        oid = self.entry_to_operon.get(entry)
        if not oid:
            return NO_CONTEXT_SCORE
        ntf = self._neighbor_tf(entry, oid)
        if not ntf:
            return NO_CONTEXT_SCORE
        ecp = self.ec_to_paths.get(ec, set())
        if not ecp:
            return 0.0
        if ec in self._neighbor_ecs(entry, oid):
            return 1.0
        return 1.0 if ecp & set(ntf.keys()) else 0.0


    def apply_binary_reject(self, preds, tau=BINARY_TAU):
        # Применение фильтра: остаются предсказания с оценкой >= tau.
        # Для предсказаний без контекста score_binary возвращает inf,
        # что всегда >= tau.
        return {e: c for e, c in preds.items()
                if c and self.score_binary(c[0][0], e) >= tau}


# GROUND TRUTH И МЕТРИКИ
```

```
def build_ground_truth(eval_df):
    # Истинные EC-номера каждого белка из оценочного набора.
    gt = defaultdict(set)
    for _, row in eval_df.iterrows():
        ec = normalize_ec(str(row["EC number"]))
        if is_full_ec(ec):
            gt[row["Entry"]].add(ec)
    return dict(gt)



def metrics(preds, gt, entries, lvl=4, bl=None):
    """Расчёт метрик качества предсказания на заданном уровне иерархии EC.

    Возвращает accuracy, F1 macro и coverage. Параметр bl (без фильтра)
    позволяет вычислить долю белков, отфильтрованных относительно
    исходного набора предсказаний.
    """
    yt, yp = [], []
    nt, np_, nc = len(entries), 0, 0
    for e in entries:
        g = gt.get(e, set())
        if not g: continue
        # Истинные EC, обрезанные до заданного уровня иерархии.
        gt_t = set(ec_at_level(x, lvl) for x in g)
        gl = sorted(gt_t)[0]
        c = preds.get(e, [])
        if not c: continue
        # Берётся top-1 предсказание (наиболее уверенное).
        pl = ec_at_level(c[0][0], lvl)
        yt.append(gl)
        yp.append(pl)
        np_ += 1
        if pl in gt_t:
            nc += 1
    if np_ == 0:
        return {"n_total": nt, "n_predicted": 0, "n_correct": 0,
                "accuracy": 0, "f1_macro": 0, "coverage": 0,
                "fraction_filtered": 0}
    from sklearn.metrics import f1_score
    f1 = f1_score(yt, yp, labels=sorted(set(yt+yp)),
                  average="macro", zero_division=0)
    fr = 0
    if bl:
        nb = sum(1 for e in entries if e in bl and bl[e])
        if nb:
            fr = 1 - np_/nb
    return {"n_total": nt, "n_predicted": np_, "n_correct": nc,
            "accuracy": nc/np_, "f1_macro": f1,
            "coverage": np_/nt if nt else 0, "fraction_filtered": fr}



def save_plot(fig, name):
    path = os.path.join(RESULTS_DIR, name)
    fig.tight_layout()
    fig.savefig(path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"  -> {name}")



# ВЫБОР ОПТИМАЛЬНОГО ПОРОГА
def find_best_threshold(sweep):
    """Возвращает порог, максимизирующий F1 macro после  фильтра.
```

```
    Критерий напрямую соответствует цели работы - повышению точности
    предсказания EC-номеров через оперонный функциональный контекст.
    Применяется одинаково к обеим моделям.
    """
    best_idx = max(range(len(sweep)),
                   key=lambda i: sweep[i]["f1_binary"])
    r = sweep[best_idx]
    return r["threshold"], r["f1_binary"], best_idx



# MAIN

def main():
    print(f"\n{'#'*70}")
    print(f"# DIPLOMA EVALUATION (критерий: max F1 после фильтра)")
    print(f"{'#'*70}")
    os.makedirs(RESULTS_DIR, exist_ok=True)


    train = load_all_train()
    save_train_fasta(train)
    eval_df = build_eval_dataset(train)
    eval_df = run_mmseqs2(eval_df)


    ground_truth = build_ground_truth(eval_df)
    pi_preds = parse_pi_predictions(PI_PRED_PATH)
    clean_preds = parse_clean_predictions(CLEAN_PRED_PATH)


    # Сопоставление белка с его стратой гомологии для последующего
    # стратифицированного анализа.
    etg = (eval_df.drop_duplicates("Entry")
                  .set_index("Entry")["identity_group"].to_dict())
    all_e = set(ground_truth.keys())
    id_groups = set(etg.values())


    scorer = OpeRONScorer(eval_df, BLACKLIST_TOP_N)
    cache = PredictionCache(pi_preds, clean_preds, all_e, scorer)


    def ent_for(st):
        if st == "ALL":
            return all_e
        return set(e for e, g in etg.items() if g == st) & all_e



    # РАЗДЕЛ 2.1: БАЗОВАЯ ОЦЕНКА + ВЫБОР ОПТИМАЛЬНОГО ПОРОГА
    print(f"\n{'#'*70}\n# 2.1 БАЗОВАЯ ОЦЕНКА PI И CLEAN + ВЫБОР ПОРОГА\n"
          f"{'#'*70}")


    # Сквозное сканирование по порогу уверенности для обеих моделей.
    # На каждом значении считаются метрики до и после применения
    # фильтра, что позволяет одновременно оценить и базовое качество модели,
    # и эффективность оперонного контекста.
    print(f"\n  ProteInfer threshold sweep:")
    pi_sweep = []
    for thr in CONF_THRESHOLDS:
        m = metrics(cache.pi(thr), ground_truth, all_e)
        m_bin = metrics(cache.pi_bin(thr), ground_truth, all_e,
                        bl=cache.pi(thr))
        pi_sweep.append({"threshold": thr, "model": "PI", **m,
                         "f1_binary": m_bin["f1_macro"],
                         "cov_binary": m_bin["coverage"]})
        print(f"    PI(>={thr:.2f}): F1={m['f1_macro']:.4f}, "
              f"cov={m['coverage']:.3f} | "
```

```
          f"+фильтр: F1={m_bin['f1_macro']:.4f}, "
          f"cov={m_bin['coverage']:.3f}")

print(f"\n  CLEAN threshold sweep:")
clean_sweep = []
for thr in CONF_THRESHOLDS:
    m = metrics(cache.cl(thr), ground_truth, all_e)
    m_bin = metrics(cache.cl_bin(thr), ground_truth, all_e,
                    bl=cache.cl(thr))
    clean_sweep.append({"threshold": thr, "model": "CLEAN", **m,
                        "f1_binary": m_bin["f1_macro"],
                        "cov_binary": m_bin["coverage"]})
    print(f"    CLEAN(>={thr:.2f}): F1={m['f1_macro']:.4f}, "
          f"cov={m['coverage']:.3f} | "
          f"+фильтр: F1={m_bin['f1_macro']:.4f}, "
          f"cov={m_bin['coverage']:.3f}")

print(f"\n  --- ВЫБОР ОПТИМАЛЬНЫХ ПОРОГОВ ---")
print(f"  Критерий: max F1 macro после оперонного фильтра")

best_pi_thr, _, pi_best_idx = find_best_threshold(pi_sweep)
best_pi = pi_sweep[pi_best_idx]

if CLEAN_FIXED_THR is None:
    best_cl_thr, _, cl_best_idx = find_best_threshold(clean_sweep)
else:
    best_cl_thr = CLEAN_FIXED_THR
    cl_best_idx = next(i for i, r in enumerate(clean_sweep)
                       if abs(r["threshold"] - best_cl_thr) < 1e-9)
best_cl = clean_sweep[cl_best_idx]

# Подробный вывод результатов сканирования с отметкой выбранного порога.
print(f"\n  Анализ выбора порога PI:")
print(f"  {'Порог':<8} {'F1 без фильтра':>16} {'F1+фильтр':>12} "
      f"{'Покрытие без фильтра':>22} {'Покрытие+фильтр':>17} {'DF1+фильтр':>12}")
print(f"  {'-'*82}")
prev_f1bin = None
for r in pi_sweep:
    delta = (f"{r['f1_binary'] - prev_f1bin:+.4f}"
             if prev_f1bin is not None else "-")
    marker = " <- ВЫБРАН" if r["threshold"] == best_pi_thr else ""
    print(f"  {r['threshold']:<8.2f} {r['f1_macro']:>8.4f} "
          f"{r['f1_binary']:>8.4f} {r['coverage']:>8.3f} "
          f"{r['cov_binary']:>8.3f} {delta:>8}{marker}")
    prev_f1bin = r["f1_binary"]

print(f"\n  Анализ выбора порога CLEAN:")
print(f"  {'Порог':<8} {'F1 без фильтра':>16} {'F1+фильтр':>12} "
      f"{'Покрытие без фильтра':>22} {'Покрытие+фильтр':>17} "
      f"{'DF1+фильтр':>12}")
print(f"  {'-'*82}")
prev_f1bin = None
for r in clean_sweep:
    delta = (f"{r['f1_binary'] - prev_f1bin:+.4f}"
             if prev_f1bin is not None else "-")
    marker = " <- ВЫБРАН" if r["threshold"] == best_cl_thr else ""
    print(f"  {r['threshold']:<8.2f} {r['f1_macro']:>8.4f} "
          f"{r['f1_binary']:>8.4f} {r['coverage']:>8.3f} "
          f"{r['cov_binary']:>8.3f} {delta:>8}{marker}")
    prev_f1bin = r["f1_binary"]

print(f"\n  Оптимальные пороги (критерий: max F1 macro после фильтра):")
print(f"    ProteInfer:  tau = {best_pi_thr:.2f}")
print(f"      F1 без фильтра:    {best_pi['f1_macro']:.4f}")
```

```
print(f"      F1 + фильтр:    {best_pi['f1_binary']:.4f}")
print(f"      Покрытие + фильтр: {best_pi['cov_binary']:.3f}")
print(f"    CLEAN:       tau = {best_cl_thr:.2f}")
print(f"      F1 без фильтра:    {best_cl['f1_macro']:.4f}")
print(f"      F1 + фильтр:    {best_cl['f1_binary']:.4f}")
print(f"      Покрытие + фильтр : {best_cl['cov_binary']:.3f}")


# Сравнение моделей при выбранных порогах позволяет обосновать выбор
# основной модели для всех последующих анализов.
delta_models = best_pi['f1_binary'] - best_cl['f1_binary']
print(f"\n  Сравнение моделей при выбранных порогах:")
print(f"    ProteInfer (tau={best_pi_thr}): "
      f" F1 без фильтра ={best_pi['f1_macro']:.4f}, "
      f" F1 + фильтр ={best_pi['f1_binary']:.4f}")
print(f"    CLEAN     (tau={best_cl_thr}): "
      f" F1 без фильтра ={best_cl['f1_macro']:.4f}, "
      f" F1 + фильтр ={best_cl['f1_binary']:.4f}")
if delta_models > 0:
    print(f"    -> ProteInfer превосходит CLEAN на "
          f"DF1+bin = {delta_models:+.4f}.")
    print(f"       В дальнейших разделах основной моделью "
          f"выбран ProteInfer.")
else:
    print(f"    -> CLEAN превосходит ProteInfer на "
          f"DF1+фильтр = {-delta_models:+.4f}.")


df_sweep = pd.DataFrame(pi_sweep + clean_sweep)
df_sweep.to_csv(os.path.join(RESULTS_DIR,
    "sec21_threshold_sweep.csv"), index=False)
print(f"  -> sec21_threshold_sweep.csv")


threshold_choice = pd.DataFrame([
    {"model": "ProteInfer",
     "optimal_threshold": best_pi_thr,
     "f1_baseline": best_pi["f1_macro"],
     "f1_binary": best_pi["f1_binary"],
     "coverage_baseline": best_pi["coverage"],
     "coverage_binary": best_pi["cov_binary"],
     "selection_criterion": "максимум F1 macro после оперонного фильтра"},
    {"model": "CLEAN",
     "optimal_threshold": best_cl_thr,
     "f1_baseline": best_cl["f1_macro"],
     "f1_binary": best_cl["f1_binary"],
     "coverage_baseline": best_cl["coverage"],
     "coverage_binary": best_cl["cov_binary"],
     "selection_criterion": "максимум F1 macro после оперонного фильтра"},
])
threshold_choice.to_csv(os.path.join(RESULTS_DIR,
    "sec21_optimal_threshold.csv"), index=False)
print(f"  -> sec21_optimal_threshold.csv")


# Метрики обеих моделей при оптимальных порогах, разрезанные по стратам.
rows_21 = []
for name, preds, thr in [
    (f"PI(>={best_pi_thr})", cache.pi(best_pi_thr), best_pi_thr),
    (f"CLEAN(>={best_cl_thr})", cache.cl(best_cl_thr), best_cl_thr)
]:
    for st in ["ALL"] + STRATA_ORDER:
        se = ent_for(st)
        if len(se) < 10: continue
        m = metrics(preds, ground_truth, se)
        rows_21.append({"model": name, "stratum": st,
                        "threshold": thr, **m})

df_21 = pd.DataFrame(rows_21)
```

```
df_21.to_csv(os.path.join(RESULTS_DIR, "sec21_baselines.csv"),
             index=False)


print(f"\n  {'Модель':<15} {'Страта':<8} "
      f"{'F1':>7} {'Acc':>7} {'Cov':>7}")
print(f"  {'-'*50}")
for _, r in df_21[df_21.stratum == "ALL"].iterrows():
    print(f"  {r.model:<15} {'ALL':<8} "
          f"{r.f1_macro:>7.4f} {r.accuracy:>7.4f} {r.coverage:>7.3f}")


# График 2.1a: сравнение моделей по стратам
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
strata_plot = ["ALL"] + [s for s in STRATA_ORDER if s in id_groups]


model_names = [f"PI(>={best_pi_thr})", f"CLEAN(>={best_cl_thr})"]
model_colors = ["#2196F3", "#FF9800"]


for idx, (metric_name, metric_label) in enumerate(
        [("f1_macro", "F1 macro"), ("coverage", "Coverage")]):
    ax = axes[idx]
    x = np.arange(len(strata_plot))
    w = 0.35
    for i, (model, color) in enumerate(zip(model_names, model_colors)):
        vals = []
        for s in strata_plot:
            sub = df_21[(df_21.model == model) & (df_21.stratum == s)]
            vals.append(sub[metric_name].values[0]
                        if len(sub) > 0 else 0)
        bars = ax.bar(x + (i - 0.5) * w, vals, w, label=model,
                      color=color, alpha=0.85,
                      edgecolor="black", linewidth=0.5)
        for bar, v in zip(bars, vals):
            ax.text(bar.get_x() + bar.get_width()/2,
                    bar.get_height() + 0.01,
                    f"{v:.3f}", ha="center", fontsize=7)
    ax.set_xticks(x)
    ax.set_xticklabels(strata_plot)
    ax.set_ylabel(metric_label)
    ax.legend(fontsize=9)
    ax.grid(axis="y", alpha=0.3)


fig.suptitle(f"2.1 Базовая оценка при оптимальном пороге "
             f"(max F1 после binary)", fontsize=13)
save_plot(fig, "sec21_baselines.png")


# График 2.1b: Pareto-фронт coverage / F1
fig, ax = plt.subplots(figsize=(10, 7))


ax.plot([r["coverage"] for r in pi_sweep],
        [r["f1_macro"] for r in pi_sweep],
        "o-", color="#2196F3", linewidth=2, markersize=8,
        label="PI baseline", alpha=0.6)
ax.plot([r["cov_binary"] for r in pi_sweep],
        [r["f1_binary"] for r in pi_sweep],
        "o--", color="#1565C0", linewidth=2, markersize=8,
        label="PI + binary")
for r in pi_sweep:
    ax.annotate(f"{r['threshold']:.1f}",
                (r["cov_binary"], r["f1_binary"]),
                textcoords="offset points", xytext=(5, 5),
                fontsize=7, color="#1565C0")


ax.plot([r["coverage"] for r in clean_sweep],
```

```
        [r["f1_macro"] for r in clean_sweep],
        "s-", color="#FF9800", linewidth=2, markersize=8,
        label="CLEAN baseline", alpha=0.6)
ax.plot([r["cov_binary"] for r in clean_sweep],
        [r["f1_binary"] for r in clean_sweep],
        "s--", color="#E65100", linewidth=2, markersize=8,
        label="CLEAN + binary")

# Звёзды отмечают выбранные пороги для обеих моделей.
ax.scatter([best_pi["cov_binary"]], [best_pi["f1_binary"]],
           s=300, marker="*", color="#E91E63", zorder=10,
           edgecolors="black", linewidth=1.5,
           label=f"PI: tau={best_pi_thr} (выбран)")
ax.scatter([best_cl["cov_binary"]], [best_cl["f1_binary"]],
           s=300, marker="*", color="#8E24AA", zorder=10,
           edgecolors="black", linewidth=1.5,
           label=f"CLEAN: tau={best_cl_thr} (выбран)")

ax.set_xlabel("Покрытие", fontsize=12)
ax.set_ylabel("F1 macro", fontsize=12)
ax.set_title("2.1 Pareto: PI vs CLEAN (без фильтра и с фильтром)", fontsize=13)
ax.legend(fontsize=9, loc="lower left")
ax.grid(alpha=0.3)
save_plot(fig, "sec21_pareto.png")



# РАЗДЕЛ 2.2: ФИЛЬТР - ПРИРОСТ F1 ПО СТРАТАМ
print(f"\n{'#'*70}\n# 2.2 ФИЛЬТР - DF1 ПО СТРАТАМ\n{'#'*70}")

pi_thr = best_pi_thr
print(f"\n  Используем порог PI: {pi_thr}")

strata_avail = [s for s in STRATA_ORDER if s in id_groups]
f1_before, f1_after = [], []

for st in strata_avail:
    se = ent_for(st)
    m_bl = metrics(cache.pi(pi_thr), ground_truth, se)
    m_f = metrics(cache.pi_bin(pi_thr), ground_truth, se,
                  bl=cache.pi(pi_thr))
    f1_before.append(m_bl["f1_macro"])
    f1_after.append(m_f["f1_macro"])
    d = m_f["f1_macro"] - m_bl["f1_macro"]
    print(f"  {st:>6}: F1 {m_bl['f1_macro']:.4f} -> "
          f"{m_f['f1_macro']:.4f} (DF1={d:+.4f}), "
          f"cov {m_bl['coverage']:.3f}->{m_f['coverage']:.3f}")

# График 2.2a: F1 до и после фильтра по стратам
fig, ax = plt.subplots(figsize=(10, 6))
colors_st = {"HIGH": "#4CAF50", "MEDIUM": "#FFC107",
             "LOW": "#FF5722", "NONE": "#9E9E9E"}
x = np.arange(len(strata_avail))
w = 0.35

ax.bar(x - w/2, f1_before, w, label=f"PI(>={pi_thr}) baseline",
       color="#BDBDBD", alpha=0.85, edgecolor="black", linewidth=0.5)
ax.bar(x + w/2, f1_after, w, label=f"PI(>={pi_thr}) + binary",
       color=[colors_st.get(s, "#999") for s in strata_avail],
       alpha=0.85, edgecolor="black", linewidth=0.5)

for i, (bf, ba) in enumerate(zip(f1_before, f1_after)):
    ax.text(x[i] - w/2, bf + 0.005, f"{bf:.3f}",
```

```
            ha="center", fontsize=9)
    ax.text(x[i] + w/2, ba + 0.005, f"{ba:.3f}",
            ha="center", fontsize=9, fontweight="bold")
    delta = ba - bf
    if delta > 0:
        ax.annotate(f"+{delta:.3f}", xy=(x[i] + w/2, ba + 0.025),
                    ha="center", fontsize=8,
                    color="#E91E63", fontweight="bold")


ax.set_xticks(x)
ax.set_xticklabels(strata_avail, fontsize=12)
ax.set_ylabel("F1 macro (EC L4)", fontsize=12)
ax.set_title(f"2.2 F1 до и после binary фильтра по стратам "
             f"(PI>={pi_thr})", fontsize=13)
ax.legend(fontsize=10, loc="upper right")
ax.grid(axis="y", alpha=0.3)
ax.set_ylim(0, max(f1_after) * 1.12)
save_plot(fig, "sec22_binary_delta_f1.png")


# График 2.2b: F1 по уровням EC иерархии
EC_LVLS = [1, 2, 3, 4]
strata_2x2 = ["ALL", "HIGH", "MEDIUM", "LOW"]


fig, axes = plt.subplots(2, 2, figsize=(12, 10), sharey=True)
for idx, st in enumerate(strata_2x2):
    ax = axes.flat[idx]
    se = ent_for(st)


    for preds, preds_bin, name, color in [
        (cache.pi(pi_thr), cache.pi_bin(pi_thr),
         "ProteInfer", "#2196F3"),
        (cache.cl(best_cl_thr), cache.cl_bin(best_cl_thr),
         "CLEAN", "#FF9800"),
    ]:
        f1_bl = [metrics(preds, ground_truth, se,
                         lvl=l)["f1_macro"] for l in EC_LVLS]
        f1_f = [metrics(preds_bin, ground_truth, se,
                        lvl=l)["f1_macro"] for l in EC_LVLS]
        ax.plot(EC_LVLS, f1_bl, "-", color=color, marker="o",
                markersize=5, linewidth=2,
                label=f"{name} baseline", alpha=0.85)
        ax.plot(EC_LVLS, f1_f, "--", color=color, marker="s",
                markersize=5, linewidth=2,
                label=f"{name} + binary", alpha=0.85)


    ax.set_title(f"Страта: {st}", fontsize=12, fontweight="bold")
    ax.set_xlabel("EC Level", fontsize=10)
    if idx % 2 == 0:
        ax.set_ylabel("F1 macro", fontsize=10)
    ax.set_xticks(EC_LVLS)
    ax.grid(alpha=0.3)
    if idx == 0:
        ax.legend(fontsize=8, loc="lower left")


fig.suptitle("2.2 F1 по уровням EC иерархии: baseline vs +binary",
             fontsize=14, y=1.01)
save_plot(fig, "sec22_f1_by_ec_level.png")


# Сводная таблица F1 по уровням EC для текста диплома и слайдов.
print(f"\n  --- Таблица F1 по уровням EC ---")
print(f"  {'Страта':<10} {'Модель':<30} "
      f"{'L1':>8} {'L2':>8} {'L3':>8} {'L4':>8} {'DL4':>8}")
print(f"  {'-'*80}")
```

```
for st in ["ALL", "HIGH", "MEDIUM", "LOW"]:
    se = ent_for(st)
    if len(se) < 10:
        continue
    for preds, preds_bin, name in [
        (cache.pi(pi_thr), cache.pi_bin(pi_thr), "ProteInfer"),
        (cache.cl(best_cl_thr), cache.cl_bin(best_cl_thr), "CLEAN"),
    ]:
        vals_bl = [metrics(preds, ground_truth, se,
                           lvl=l)["f1_macro"] for l in [1,2,3,4]]
        vals_bn = [metrics(preds_bin, ground_truth, se,
                           lvl=l)["f1_macro"] for l in [1,2,3,4]]
        delta_l4 = vals_bn[3] - vals_bl[3]
        row = f"  {st:<10} {name + ' (без / с фильтр.)':<30}"
        for bl, bn in zip(vals_bl, vals_bn):
            row += f" {bl:.3f}/{bn:.3f}"
        row += f"  {delta_l4:+.3f}"
        print(row)


# Метрики PI по стратам для итоговой таблицы.
print(f"\n  --- Метрики PI по стратам ---")
print(f"  {'Страта':<10} {'F1 без фильтра':>16} {'F1+фильтр':>12} {'DF1':>8} "
      f"{'Точность без фильтра':>22} {'Точность+фильтр':>17} "
      f"{'Покрытие без фильтра':>22} {'Покрытие+фильтр':>17}")
for st in ["HIGH", "MEDIUM", "LOW", "ALL"]:
    se = ent_for(st)
    m_bl = metrics(cache.pi(pi_thr), ground_truth, se)
    m_bn = metrics(cache.pi_bin(pi_thr), ground_truth, se,
                   bl=cache.pi(pi_thr))
    print(f"  {st:<10} {m_bl['f1_macro']:>8.4f} "
          f"{m_bn['f1_macro']:>8.4f} "
          f"{m_bn['f1_macro']-m_bl['f1_macro']:>+8.4f} "
          f"{m_bl['accuracy']:>8.4f} {m_bn['accuracy']:>8.4f} "
          f"{m_bl['coverage']:>8.3f} {m_bn['coverage']:>8.3f}")


# График 2.2c: тепловая карта по классам EC (первая цифра)
# Для каждого белка определяется его EC-класс верхнего уровня (1..7);
# затем считается F1 macro по классу до и после применения фильтра.
entry_ec_class = {}
for entry, ecs in ground_truth.items():
    for ec in sorted(ecs):
        entry_ec_class[entry] = ec.split(".")[0]
        break


ec_classes = sorted(set(entry_ec_class.values()), key=lambda x: int(x))
ec_labels = [f"EC {c}" for c in ec_classes]


models_data = [
    ("ProteInfer", cache.pi(pi_thr), cache.pi_bin(pi_thr)),
    ("CLEAN", cache.cl(best_cl_thr), cache.cl_bin(best_cl_thr)),
]


matrix = np.full((len(models_data), len(ec_classes)), np.nan)
f1_af_matrix = np.full((len(models_data), len(ec_classes)), np.nan)


for i, (mname, preds_bl, preds_f) in enumerate(models_data):
    for j, ec_cls in enumerate(ec_classes):
        cls_entries = set(
            e for e, c in entry_ec_class.items()
            if c == ec_cls) & all_e
        if len(cls_entries) < 10:
            continue
        m_bl = metrics(preds_bl, ground_truth, cls_entries)
```

```
        m_f = metrics(preds_f, ground_truth, cls_entries, bl=preds_bl)
        f1_af_matrix[i, j] = m_f["f1_macro"]
        matrix[i, j] = m_f["f1_macro"] - m_bl["f1_macro"]


from matplotlib.colors import TwoSlopeNorm


fig, axes = plt.subplots(1, 2, figsize=(16, 4))


ax = axes[0]
vmin = (np.nanmin(matrix) if not np.all(np.isnan(matrix)) else -0.01)
vmax = (np.nanmax(matrix) if not np.all(np.isnan(matrix)) else 0.01)
if vmin >= 0: vmin = -0.001
if vmax <= 0: vmax = 0.001
# TwoSlopeNorm с центром в 0 даёт симметричную цветовую шкалу,
# где нулевое изменение - белый, положительные - зелёный, отрицательные - красный.
norm = TwoSlopeNorm(vmin=vmin, vcenter=0, vmax=vmax)
im = ax.imshow(matrix, cmap="RdYlGn", norm=norm, aspect="auto")
ax.set_xticks(range(len(ec_labels)))
ax.set_xticklabels(ec_labels, fontsize=9)
ax.set_yticks(range(len(models_data)))
ax.set_yticklabels([m[0] for m in models_data], fontsize=11)
for i in range(len(models_data)):
    for j in range(len(ec_classes)):
        v = matrix[i, j]
        if not np.isnan(v):
            ax.text(j, i, f"{v:+.3f}", ha="center", va="center",
                    fontsize=9, fontweight="bold",
                    color="black" if abs(v) < vmax * 0.5 else "white")
ax.set_title("DF1 macro (+ binary - baseline)", fontsize=12)
fig.colorbar(im, ax=ax, label="DF1", shrink=0.8)


ax = axes[1]
im2 = ax.imshow(f1_af_matrix, cmap="RdYlGn", aspect="auto",
                vmin=0.3, vmax=0.95)
ax.set_xticks(range(len(ec_labels)))
ax.set_xticklabels(ec_labels, fontsize=9)
ax.set_yticks(range(len(models_data)))
ax.set_yticklabels([m[0] for m in models_data], fontsize=11)
for i in range(len(models_data)):
    for j in range(len(ec_classes)):
        v = f1_af_matrix[i, j]
        if not np.isnan(v):
            ax.text(j, i, f"{v:.3f}", ha="center", va="center",
                    fontsize=9, fontweight="bold",
                    color="white" if v < 0.5 else "black")
ax.set_title("F1 macro после binary фильтра", fontsize=12)
fig.colorbar(im2, ax=ax, label="F1 macro", shrink=0.8)


fig.suptitle("2.2 F1 macro по EC классам (L1): "
             "оперонный binary фильтр", fontsize=14, y=1.03)
save_plot(fig, "sec22_ec_class_heatmap.png")



# РАЗДЕЛ 2.4: ОШИБКИ ПРИ ВЫСОКОЙ УВЕРЕННОСТИ И RECALL ФИЛЬТРА


print(f"\n{'#'*70}\n# 2.4 ОШИБКИ ПРИ ВЫСОКОЙ УВЕРЕННОСТИ\n{'#'*70}")


# При оптимальном пороге собираем данные по уверенным ошибкам модели:
# confidence >= 0.9, но предсказание не совпадает с истинным EC-номером.
# Затем определяется, сколько таких ошибок отклоняет оперонный фильтр
# (recall фильтра), в разрезе по стратам гомологии.
hce_rows = []
pi_sub = cache.pi(best_pi_thr)
```

```
for e, c in pi_sub.items():
    if c[0][1] < 0.9: continue
    gt_e = ground_truth.get(e, set())
    correct = c[0][0] in gt_e
    sc = scorer.score_binary(c[0][0], e)
    # rejected = фильтр счёл предсказание противоречащим контексту
    # (score < tau). NO_CONTEXT (inf) исключается - такие предсказания
    # не отклоняются фильтром.
    rej = sc < BINARY_TAU and sc != float("inf")
    hce_rows.append({"entry": e, "confidence": c[0][1],
                     "correct": correct, "binary_score": sc,
                     "rejected": rej,
                     "stratum": etg.get(e, "ALL")})


hce_df = pd.DataFrame(hce_rows)
if len(hce_df):
    n_t = len(hce_df)
    n_w = (~hce_df.correct).sum()
    n_wr = ((~hce_df.correct) & hce_df.rejected).sum()
    print(f"\n  Уверенные ошибки (conf>=0.9): "
          f"{n_w}/{n_t} ({100*n_w/n_t:.1f}%)")
    if n_w > 0:
        print(f"  Оперонный фильтр поймал: {n_wr}/{n_w} "
              f"(recall={n_wr/n_w:.3f})")


# График 2.4: уверенные ошибки и recall фильтра по стратам
if len(hce_df):
    fig, ax = plt.subplots(figsize=(8, 5))
    sl, recall_vals, error_rates = [], [], []


    for st in STRATA_ORDER:
        sub = hce_df[hce_df.stratum == st]
        if len(sub) == 0: continue
        n_total = len(sub)
        n_wrong = (~sub.correct).sum()
        n_caught = ((~sub.correct) & sub.rejected).sum()
        if n_wrong == 0: continue
        sl.append(st)
        recall_vals.append(100 * n_caught / n_wrong)
        error_rates.append(100 * n_wrong / n_total)


    if sl:
        x = np.arange(len(sl))
        w = 0.35
        ax.bar(x - w/2, error_rates, w,
               label="Ошибки модели (% от всех белков)",
               color="#E53935", alpha=0.7,
               edgecolor="black", linewidth=0.5)
        ax.bar(x + w/2, recall_vals, w,
               label="Из них пойманы binary (% от ошибок)",
               color="#4CAF50", alpha=0.85,
               edgecolor="black", linewidth=0.5)
        for i, (er, rc) in enumerate(zip(error_rates, recall_vals)):
            ax.text(x[i] - w/2, er + 0.5, f"{er:.1f}%",
                    ha="center", fontsize=10)
            ax.text(x[i] + w/2, rc + 0.5, f"{rc:.1f}%",
                    ha="center", fontsize=10, fontweight="bold")
        ax.set_xticks(x)
        ax.set_xticklabels(sl, fontsize=12)
        ax.set_ylabel("%", fontsize=12)
        ax.set_title("2.4 Ошибки модели (conf>=0.9) и "
                     "recall binary по стратам", fontsize=13)
        ax.legend(fontsize=10)
        ax.grid(axis="y", alpha=0.3)
    save_plot(fig, "sec24_high_conf_errors.png")
```

```
    # ── Финал ──────────────────────────────────────────────────────────
    print(f"\n{'#'*70}\n# ЗАВЕРШЕНО\n{'#'*70}")
    print(f"\n  Оптимальный порог PI:    {best_pi_thr} "
          f"(критерий: max F1 после фильтра)")
    print(f"  Оптимальный порог CLEAN: {best_cl_thr} (тот же критерий)")
    print(f"\nФайлы в {RESULTS_DIR}/:")
    print(f"  sec21_threshold_sweep.csv")
    print(f"  sec21_optimal_threshold.csv")
    print(f"  sec21_baselines.csv / .png")
    print(f"  sec21_pareto.png")
    print(f"  sec22_binary_delta_f1.png")
    print(f"  sec22_f1_by_ec_level.png")
    print(f"  sec22_ec_class_heatmap.png")
    print(f"  sec24_high_conf_errors.png")


if __name__ == "__main__":
    main()
```